Trees



11

Chapter Summary

- Introduction to Trees
- Applications of Tree
- Tree Traversal
- Spanning Trees
- Minimum Spanning Trees

Tree

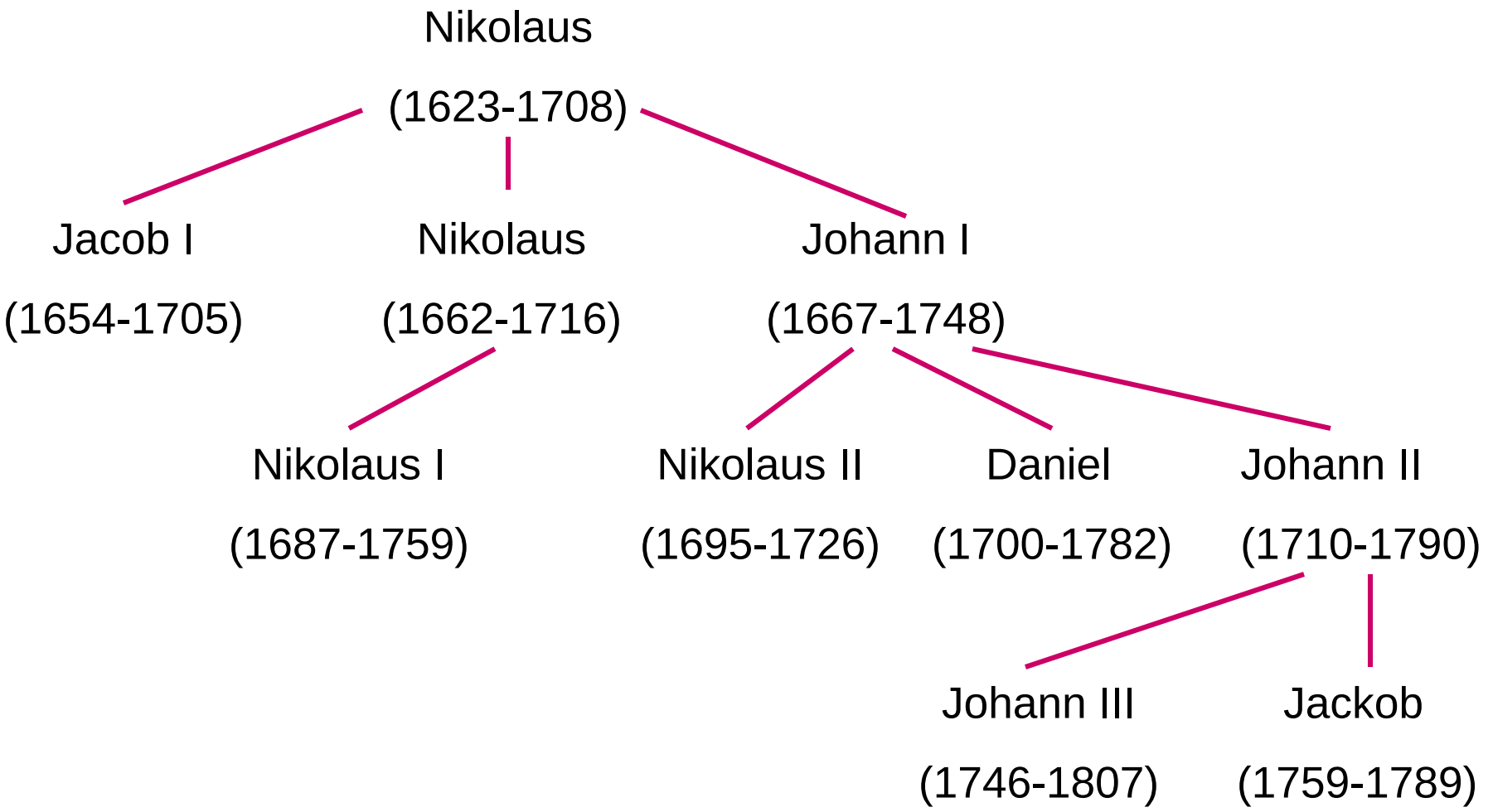
- A connected graph that contains no simple circuits is called a tree.
- Tree were used as long ago as 1857, when the English mathematician Authur Cayley used them to count certain types of chemical compounds. Since that time, trees have been employed to solve problems in a wide variety of disciplines.
- Trees are particularly useful in computer science.
  - Construct efficient algorithms for locating items in a list
  - Construct networks with the least expensive set of telephone lines linking distribute computers
  - Construct efficient codes for storing and transmitting data

11.1 Introduction to Trees

- Definition 1: A tree is a connected undirected graph with no simple circuits.

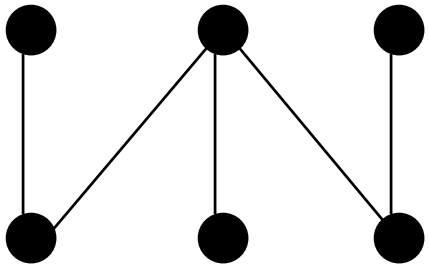
- Theorem 1: An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

The Bernoulli family of mathematicians

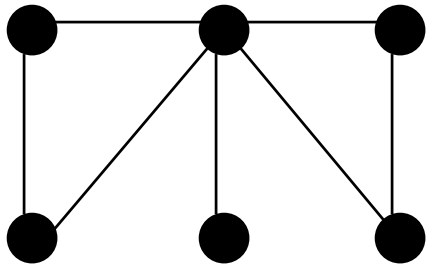


Which graphs are trees?

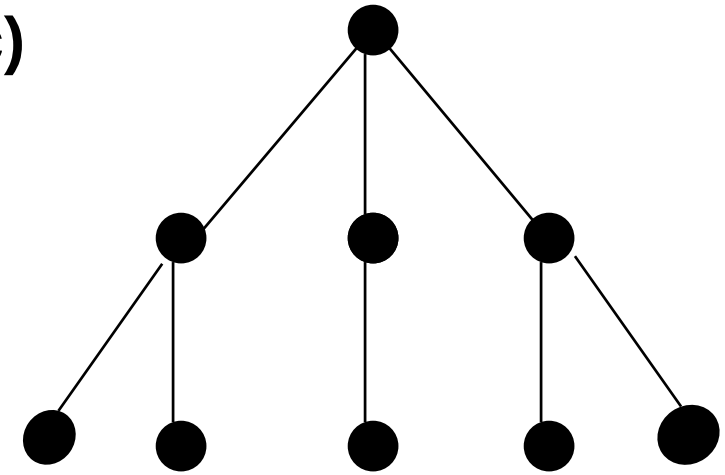
a)



b)



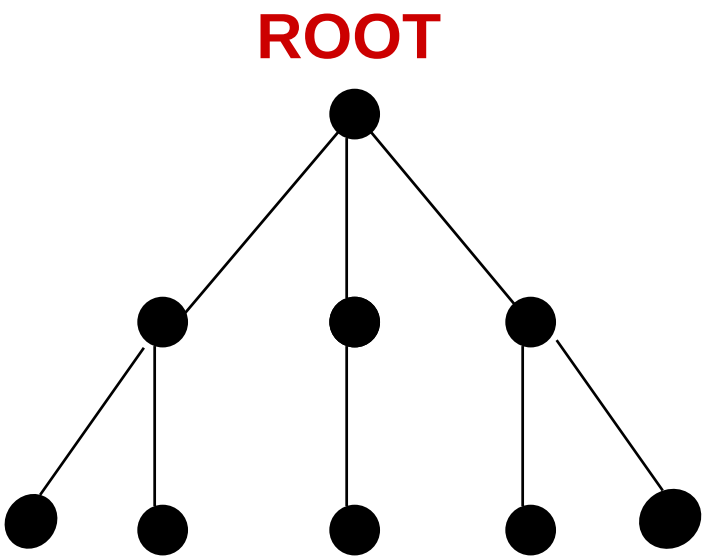
c)



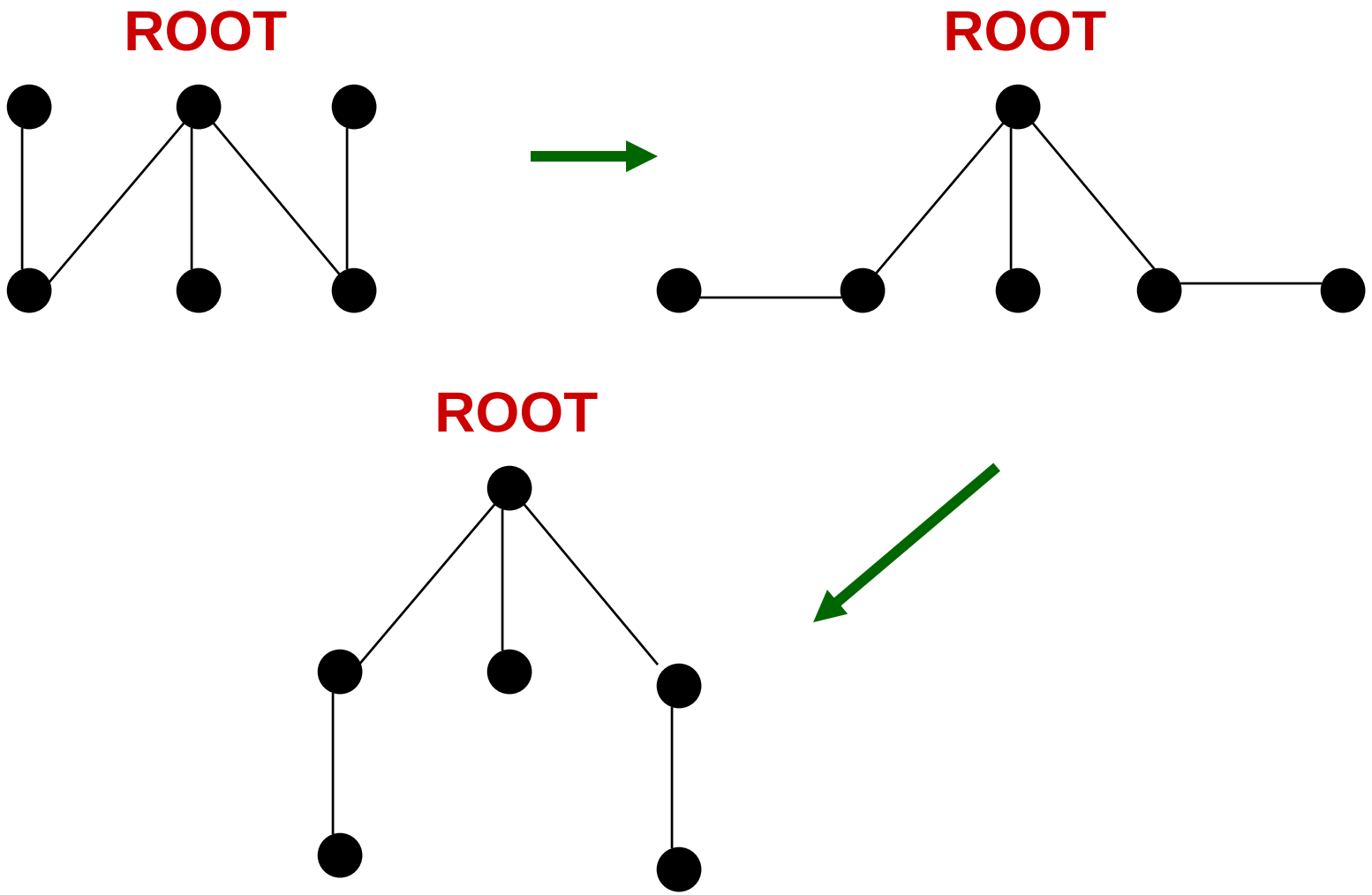
Root

- In many applications of trees a particular vertex of a tree is designated as the root.
- Once we specify a root, we direct each edge away from the root.
- Thus, a tree together with its root produces a directed graph called a rooted tree.

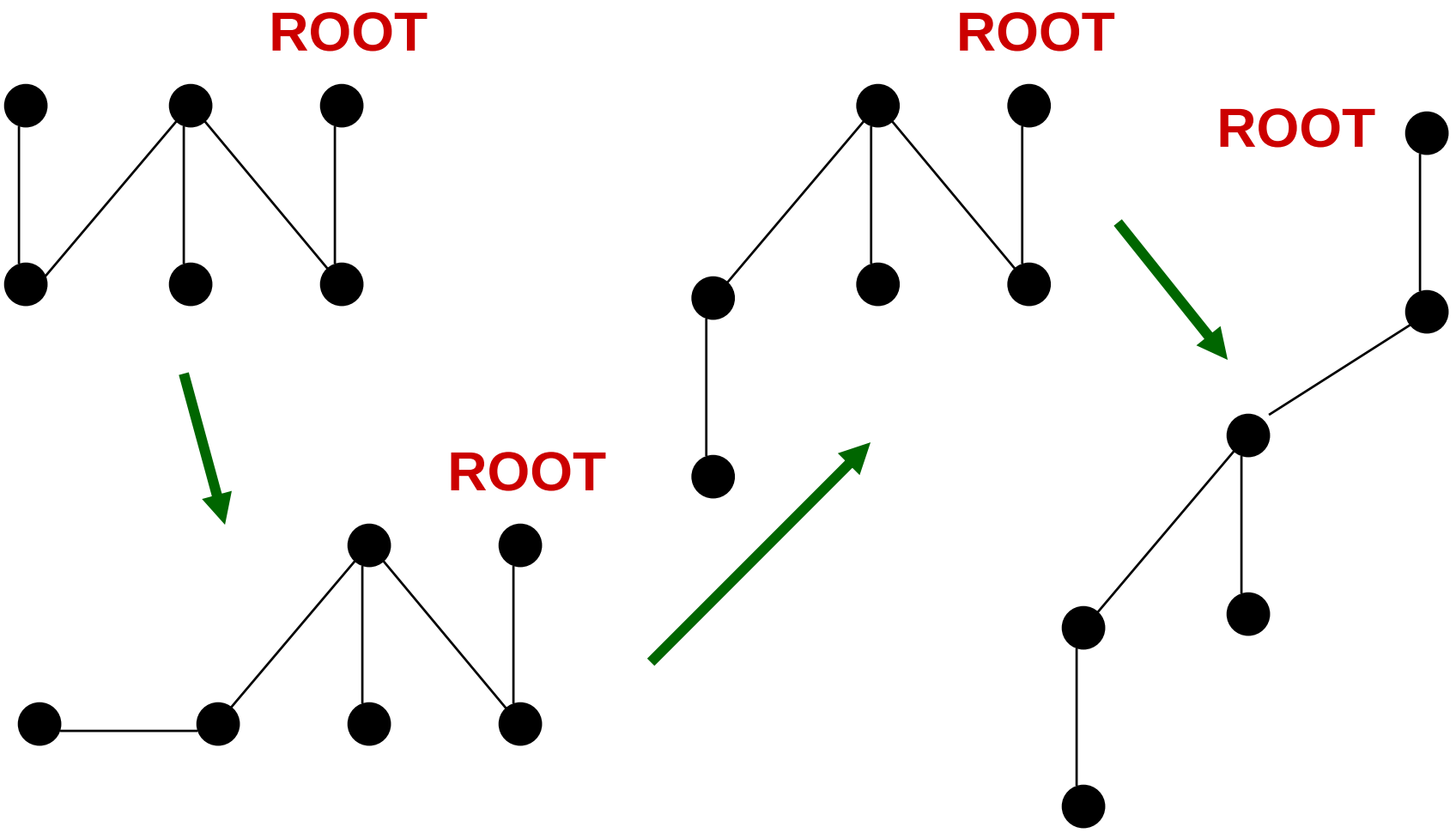
Example 1



Example 2



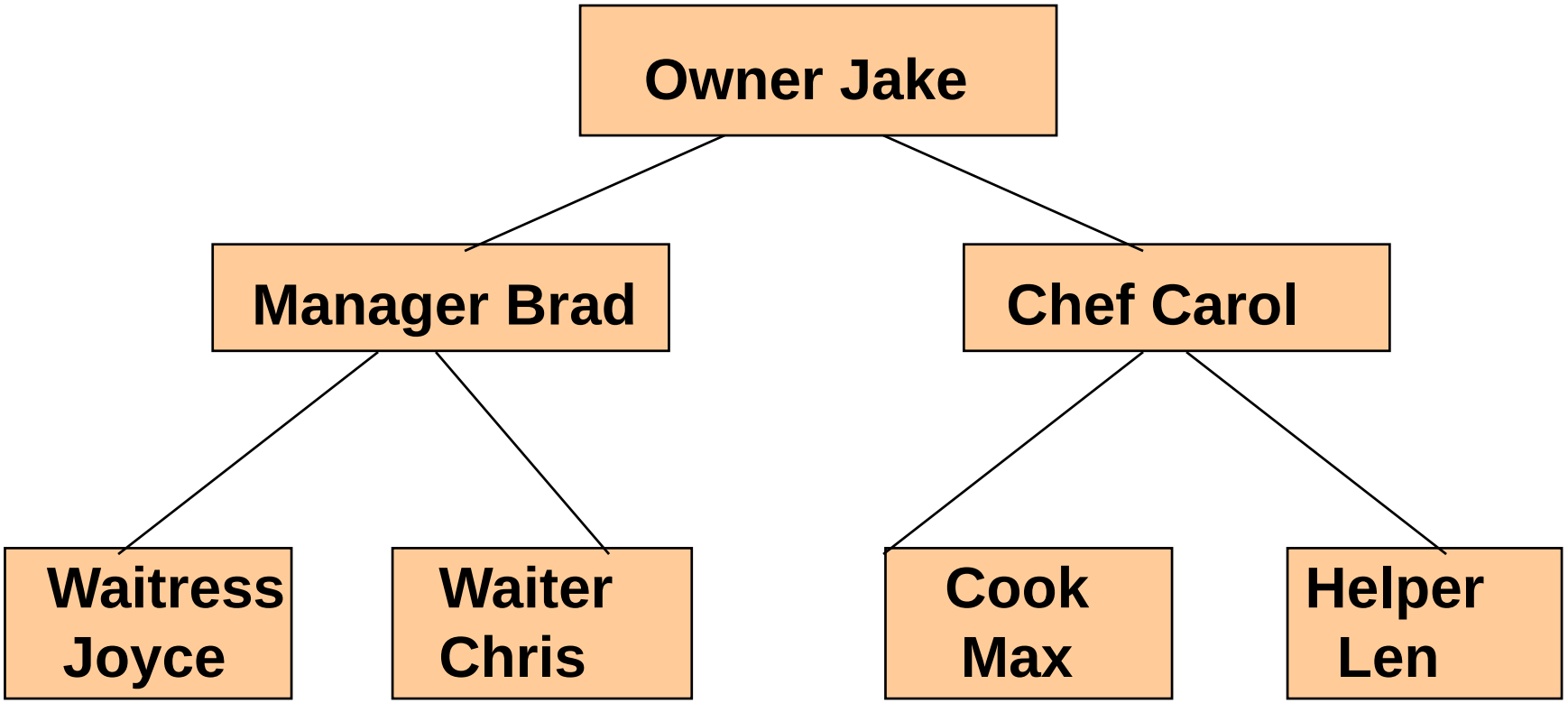
What if a different root is chosen?



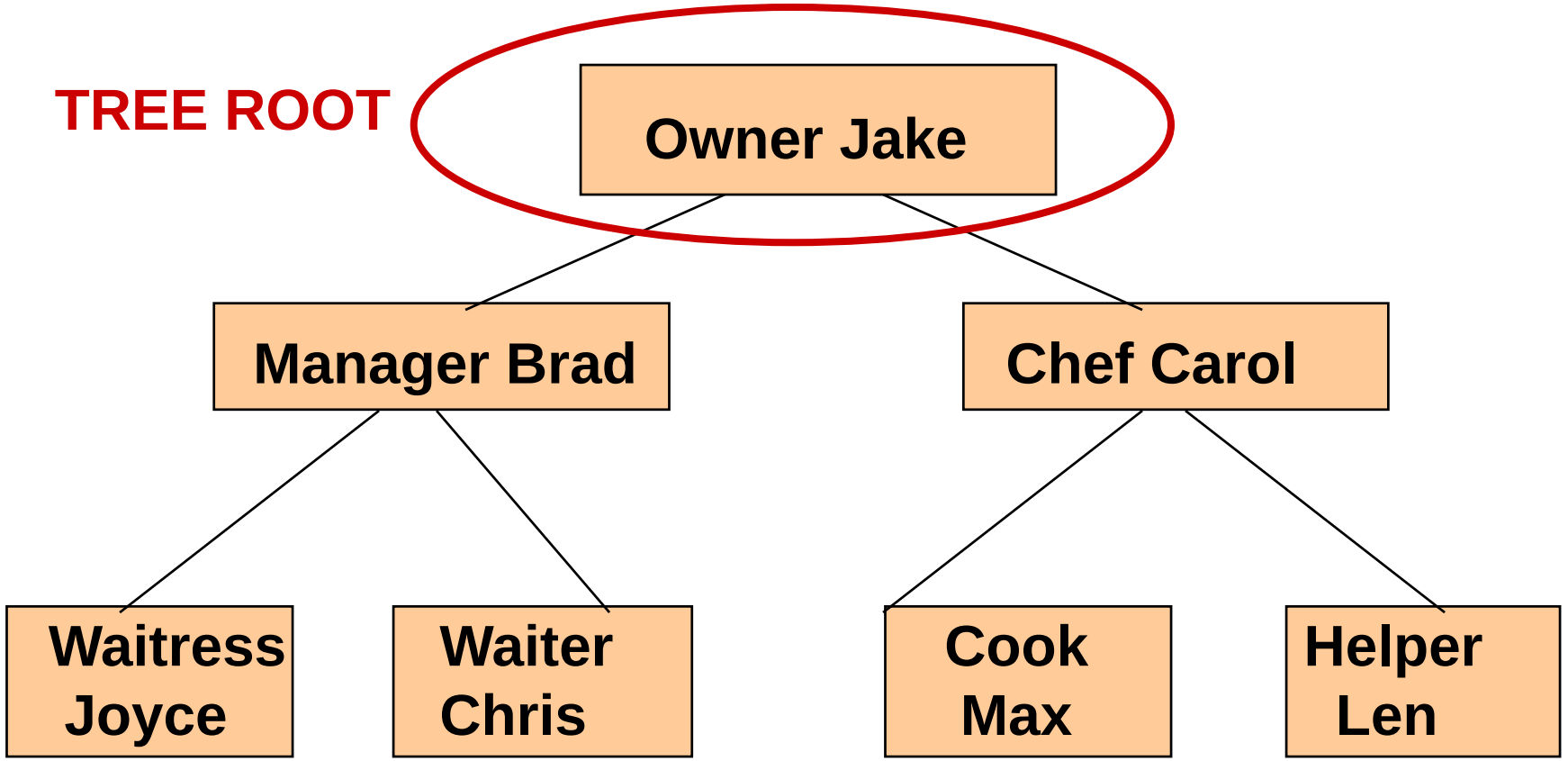
Some concepts in tree

- Parents VS. Children
- Siblings
- Ancestor VS. Descendants
- Root, leaf, and internal vertices
- Subtrees

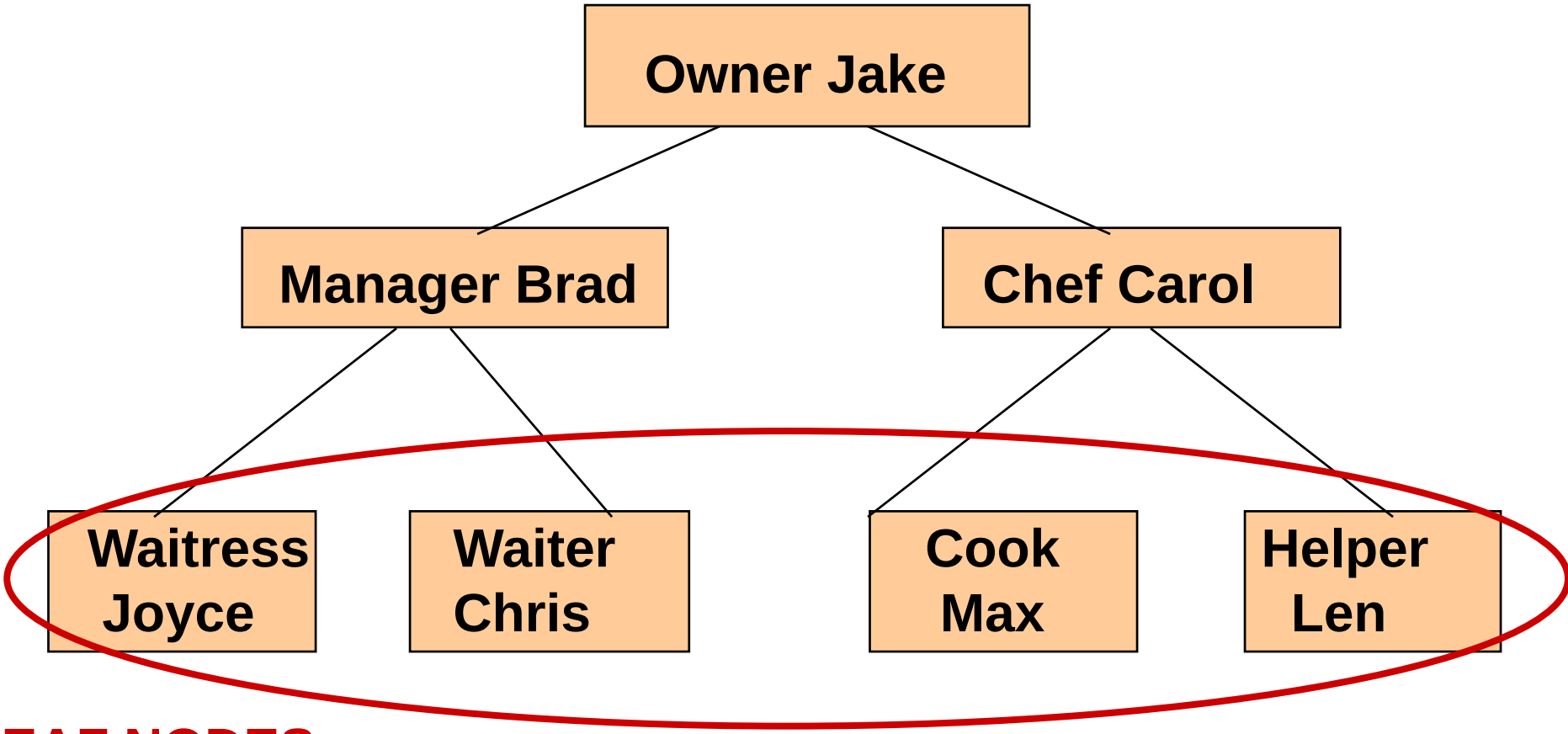
Jake's Pizza Shop Tree



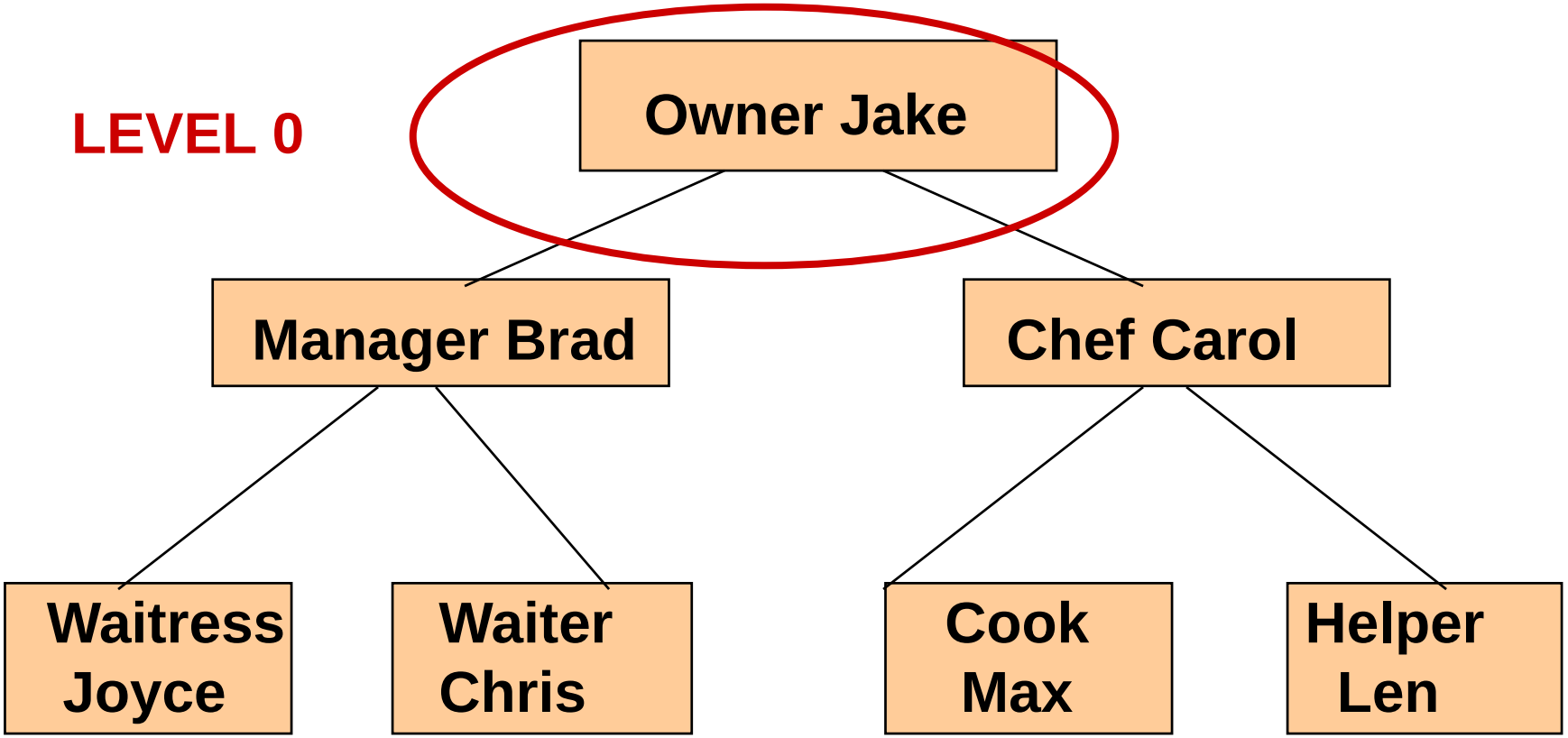
A Tree Has a Root



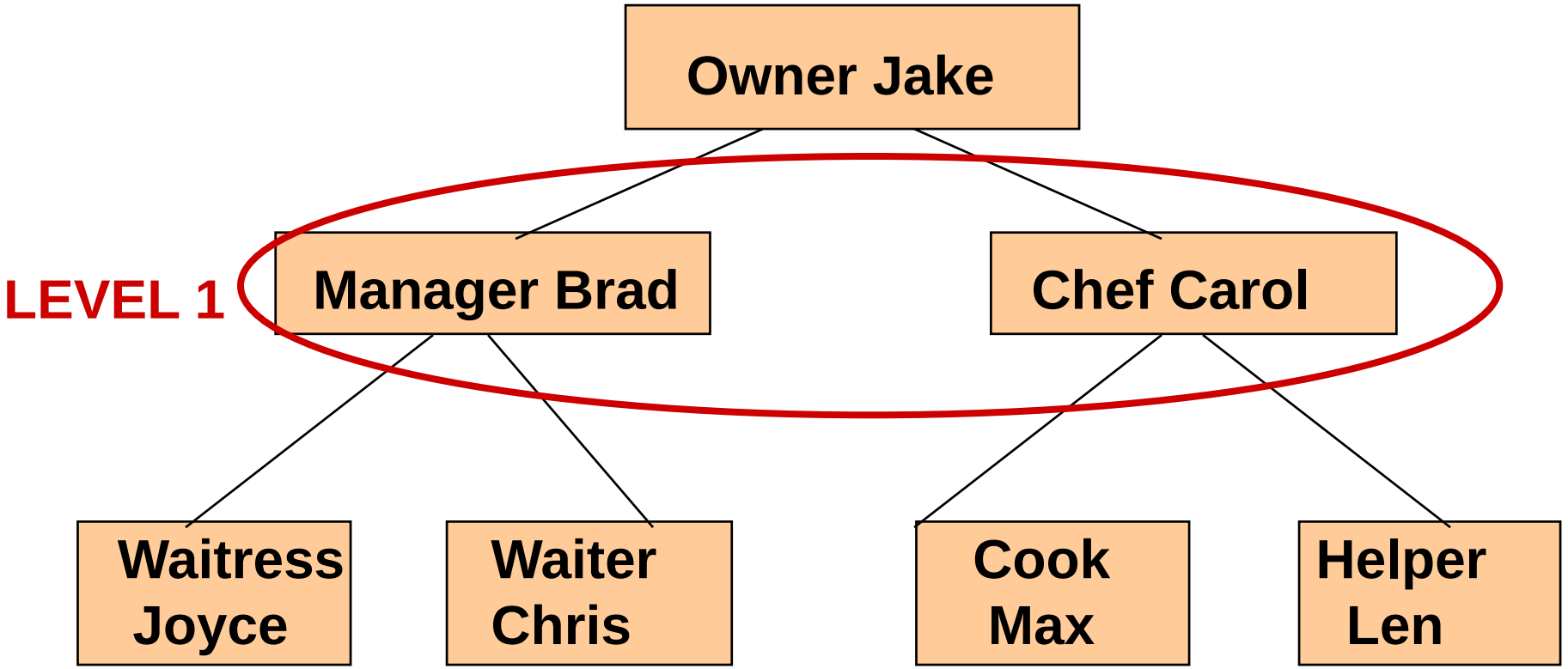
Leaf nodes have no children



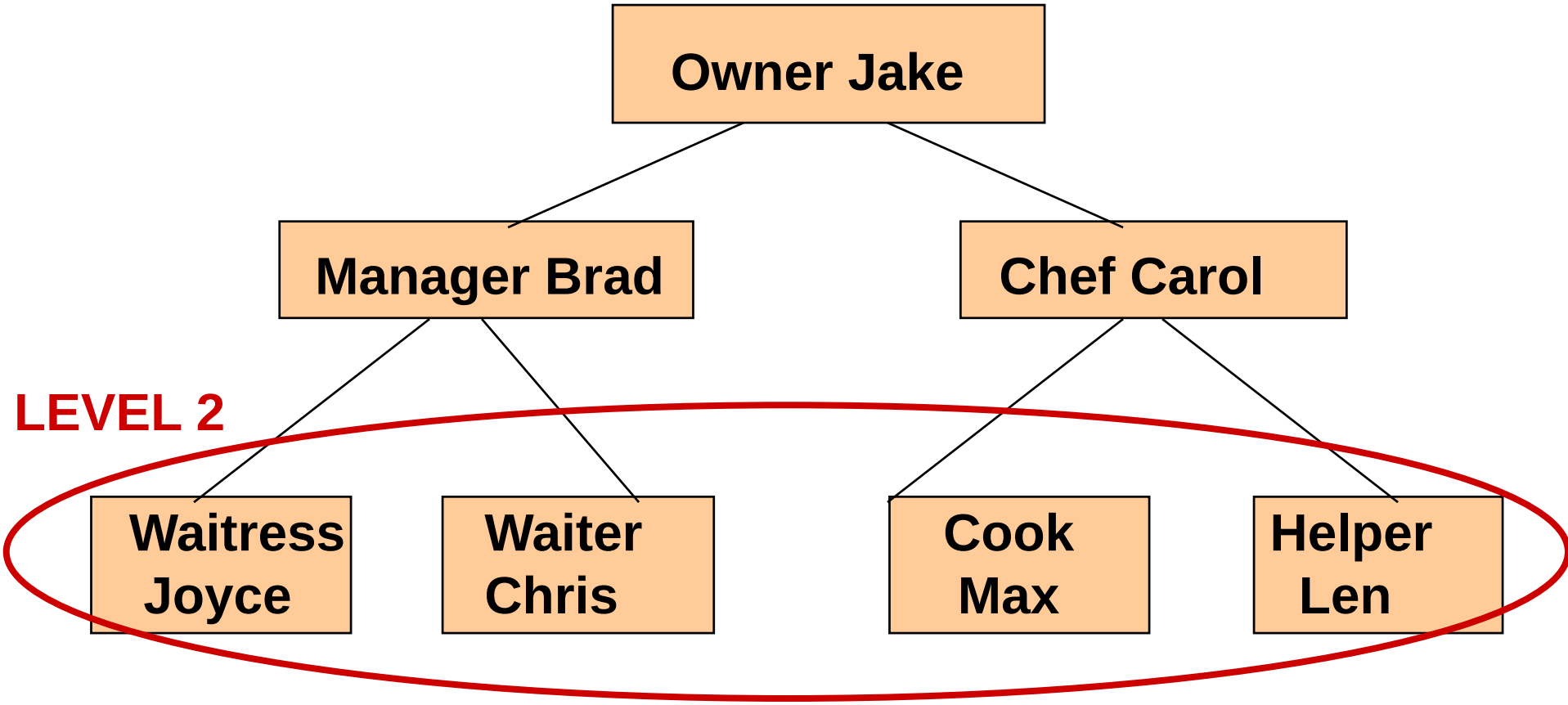
A Tree Has Levels



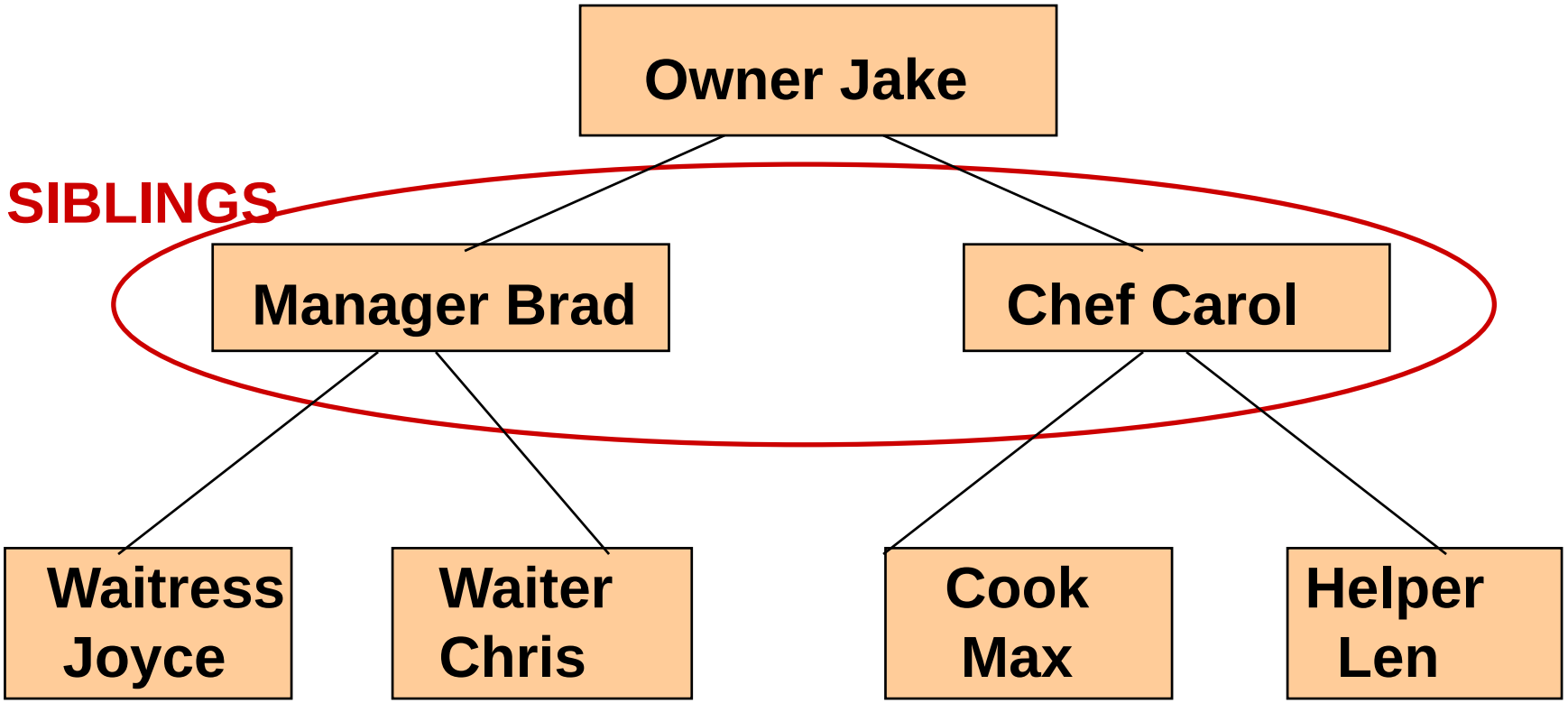
A Tree Has Levels



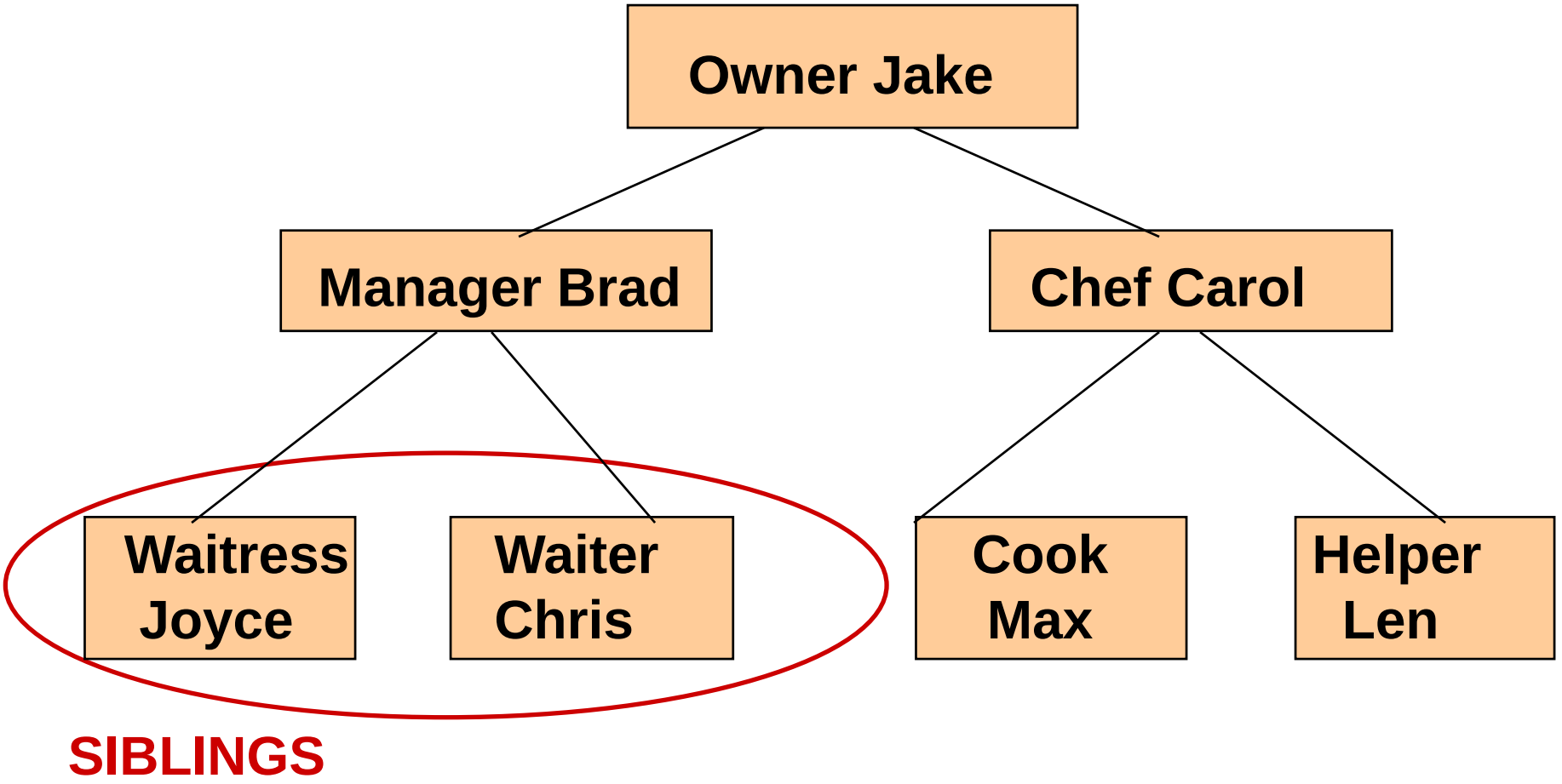
A Tree Has Levels



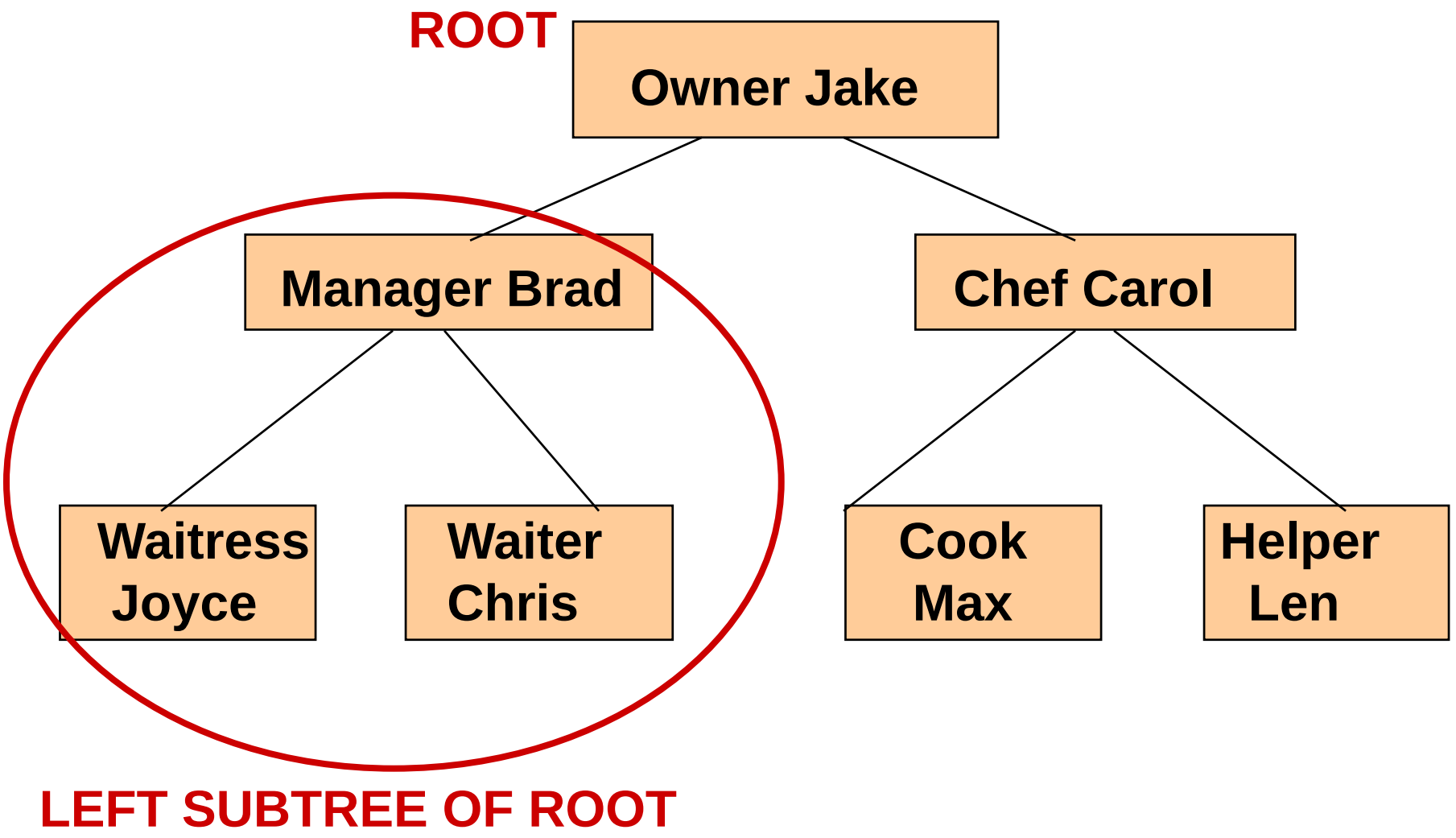
Sibling nodes have same parent



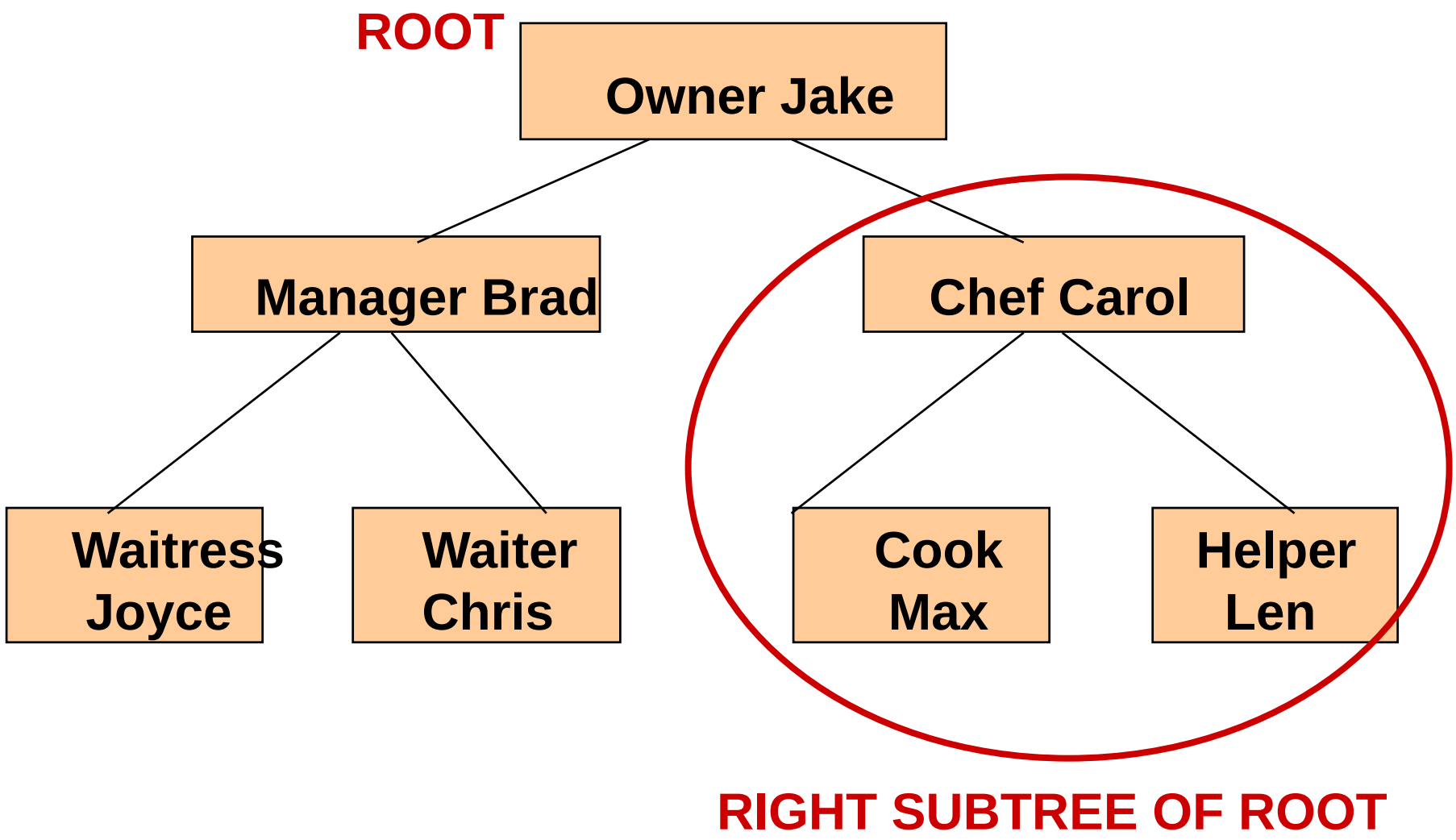
Sibling nodes have same parent



A Subtree


ROOT
Owner Jake
Manager Brad
Chef Carol
Waitress Joyce
Waiter Chris
Cook Max
Helper Len
LEFT SUBTREE OF ROOT

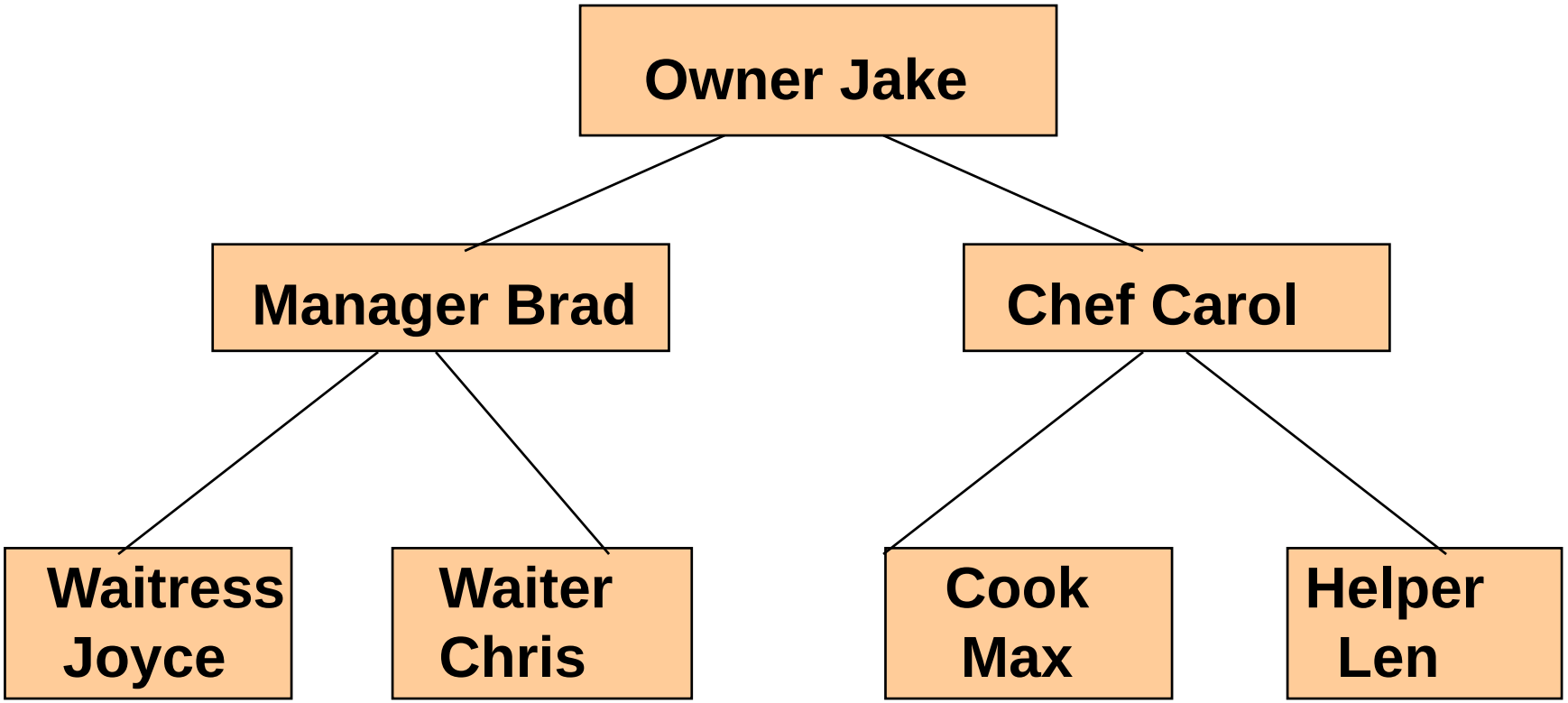
Another Subtree



Internal Vertex

- A vertex that has children is called an internal vertex.
- The subtree at vertex v is the subgraph of the tree consisting of vertex v and its descendants and all edges incident to those descendants.

How many internal vertices?



Binary Tree and Ordered Binary Tree

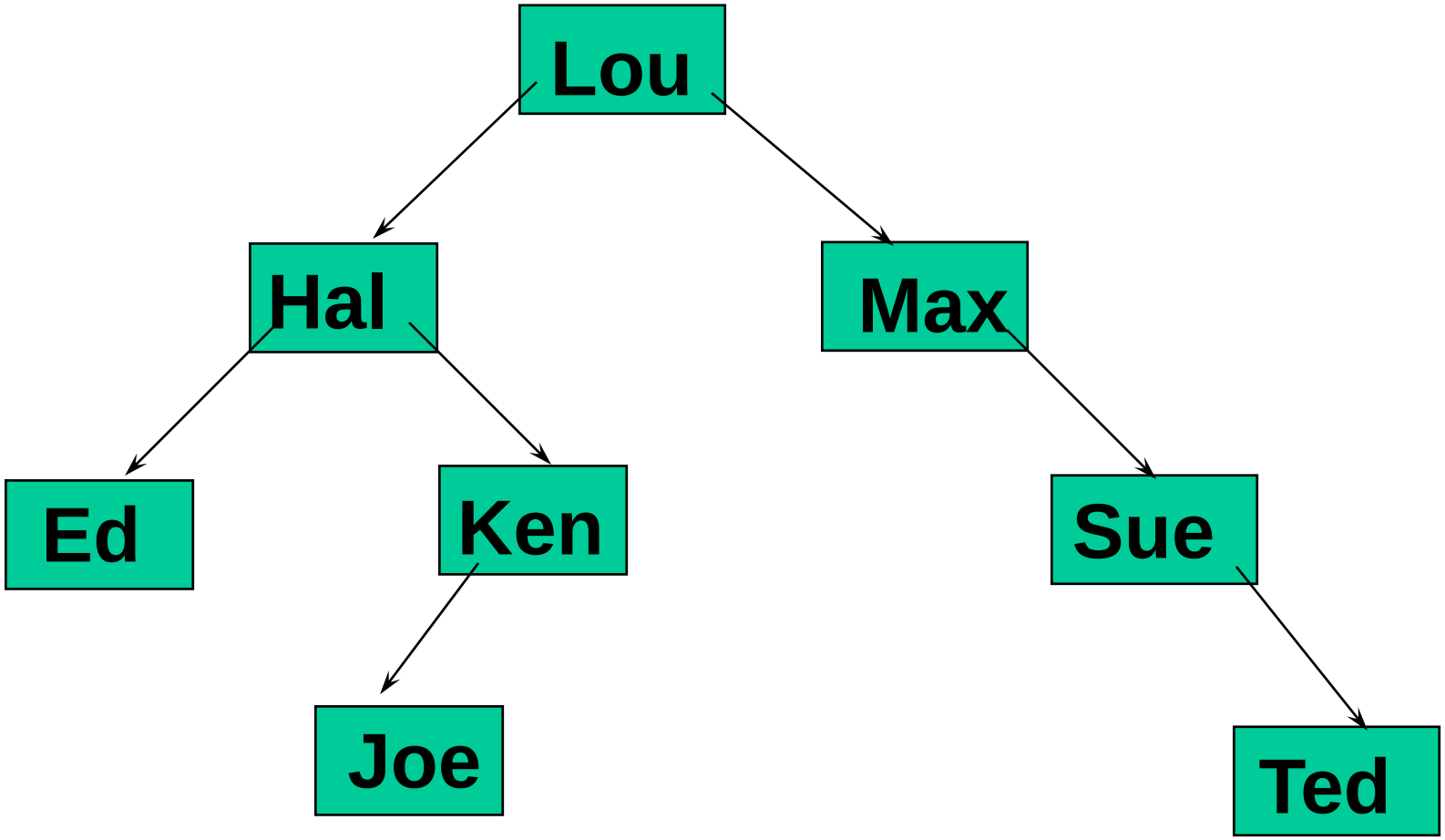
- Definition 3: A rooted tree is called an m-ary tree if every internal vertex has no more than m children.

  A rooted tree is called a binary tree if every internal vertex has no more than 2 children.
- The tree is called a full binary tree if every internal vertex has exactly 2 children.

- Definition 4: An ordered rooted tree is a rooted tree where the children of each internal vertex are ordered.

  In an ordered binary tree, the two possible children of a vertex are called the left child and the right child, if they exist.

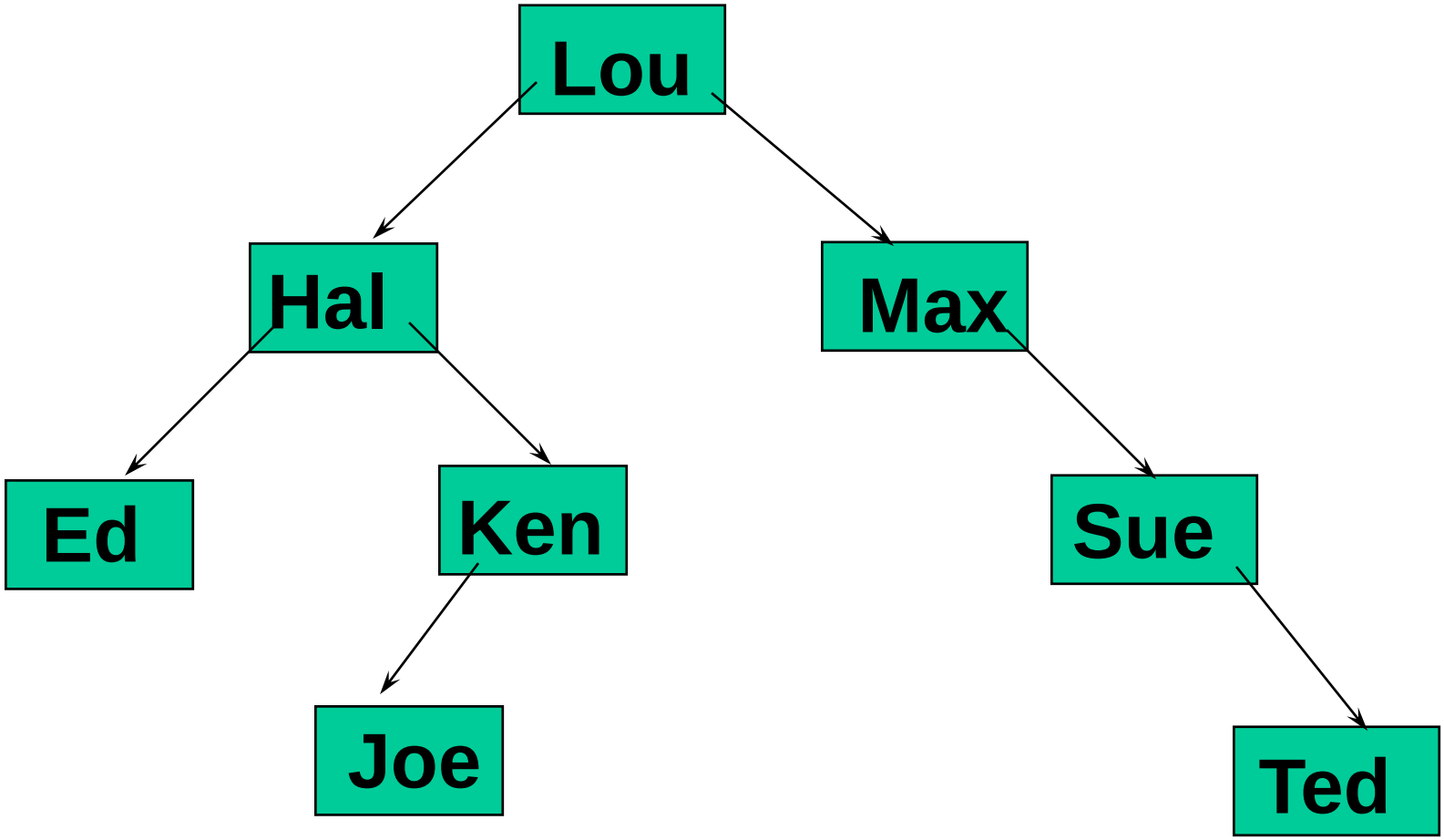
An Ordered Binary Tree



Parent

- The parent of a non-root vertex v is the unique vertex u with a directed edge from u to v.

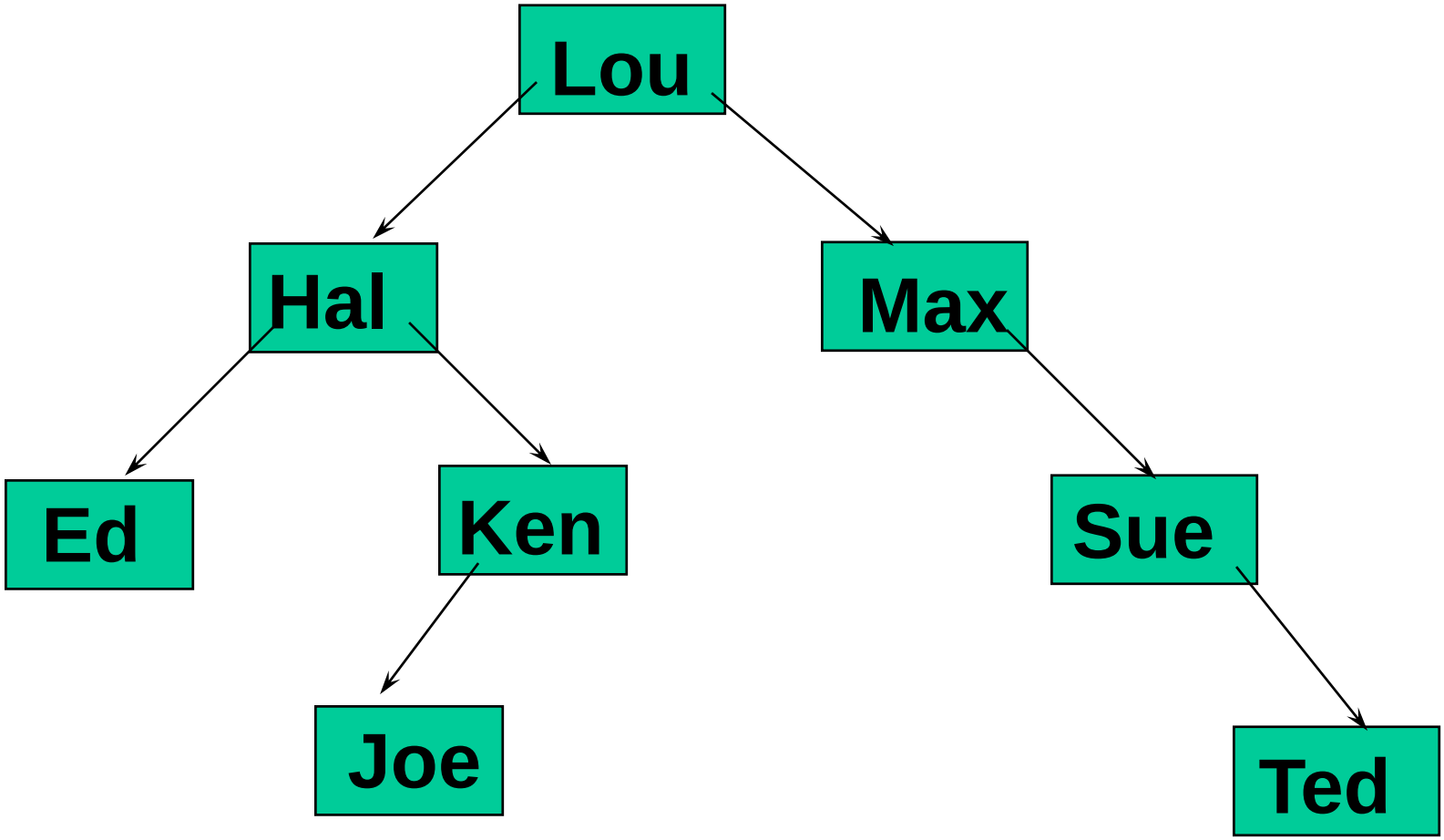
What is the parent of Ed?



Leaf

- A vertex is called a leaf if it has no children.

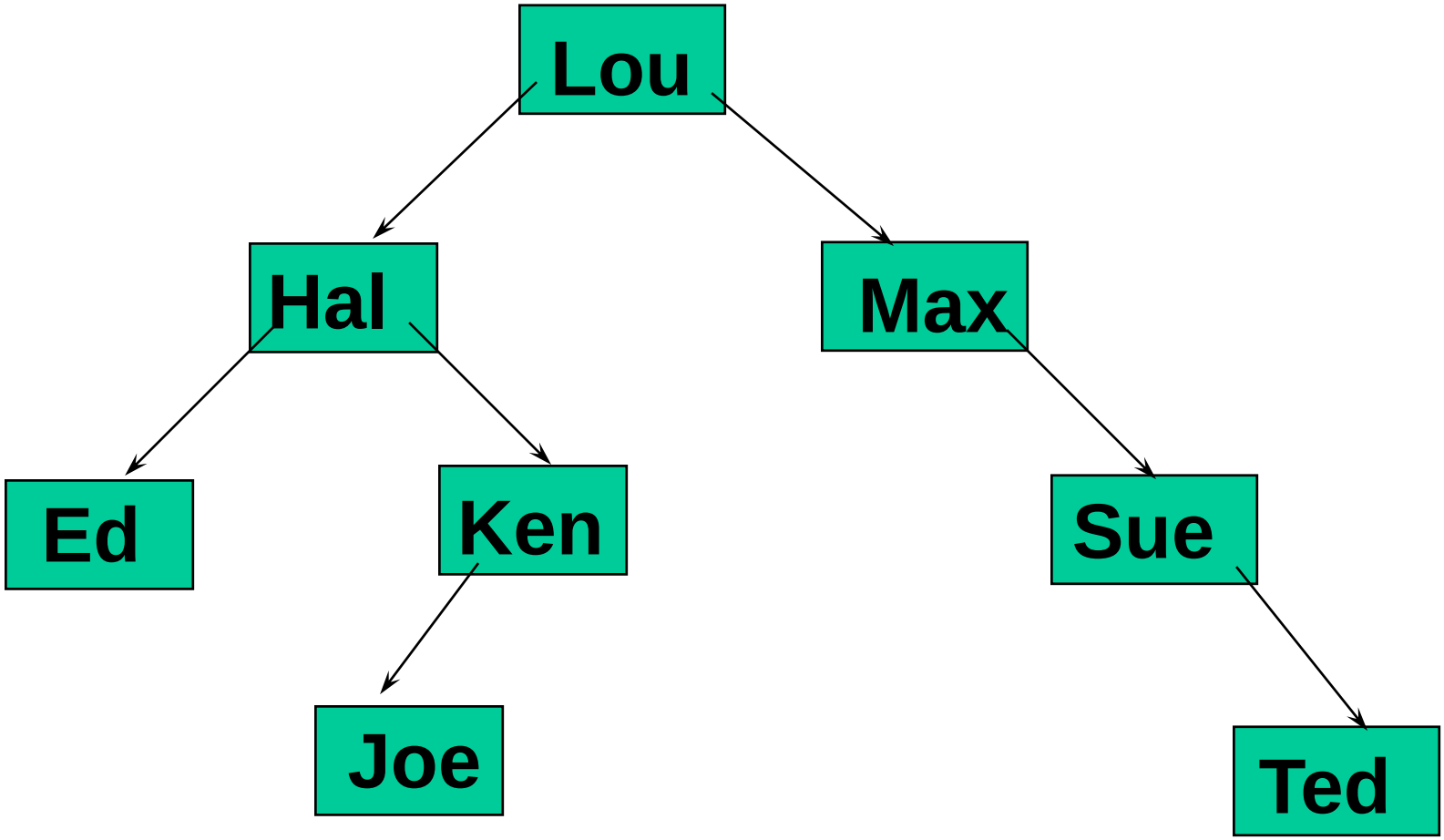
How many leaves?



Ancestors

- The ancestors of a non-root vertex are all the vertices in the path from root to this vertex.

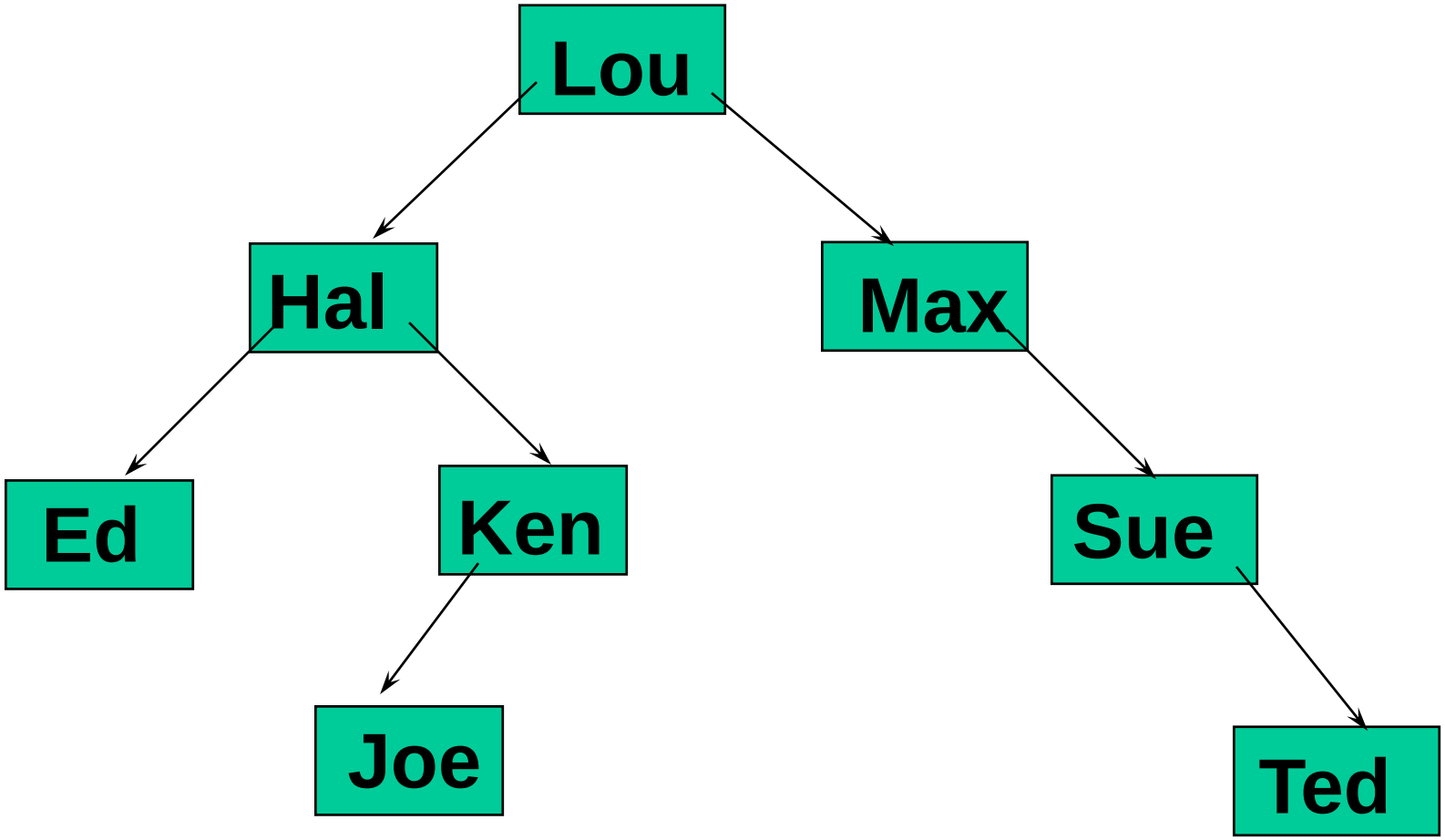
How many ancestors of Ken?



Descendants

- The descendants of vertex v are all the vertices that have v as an ancestor.

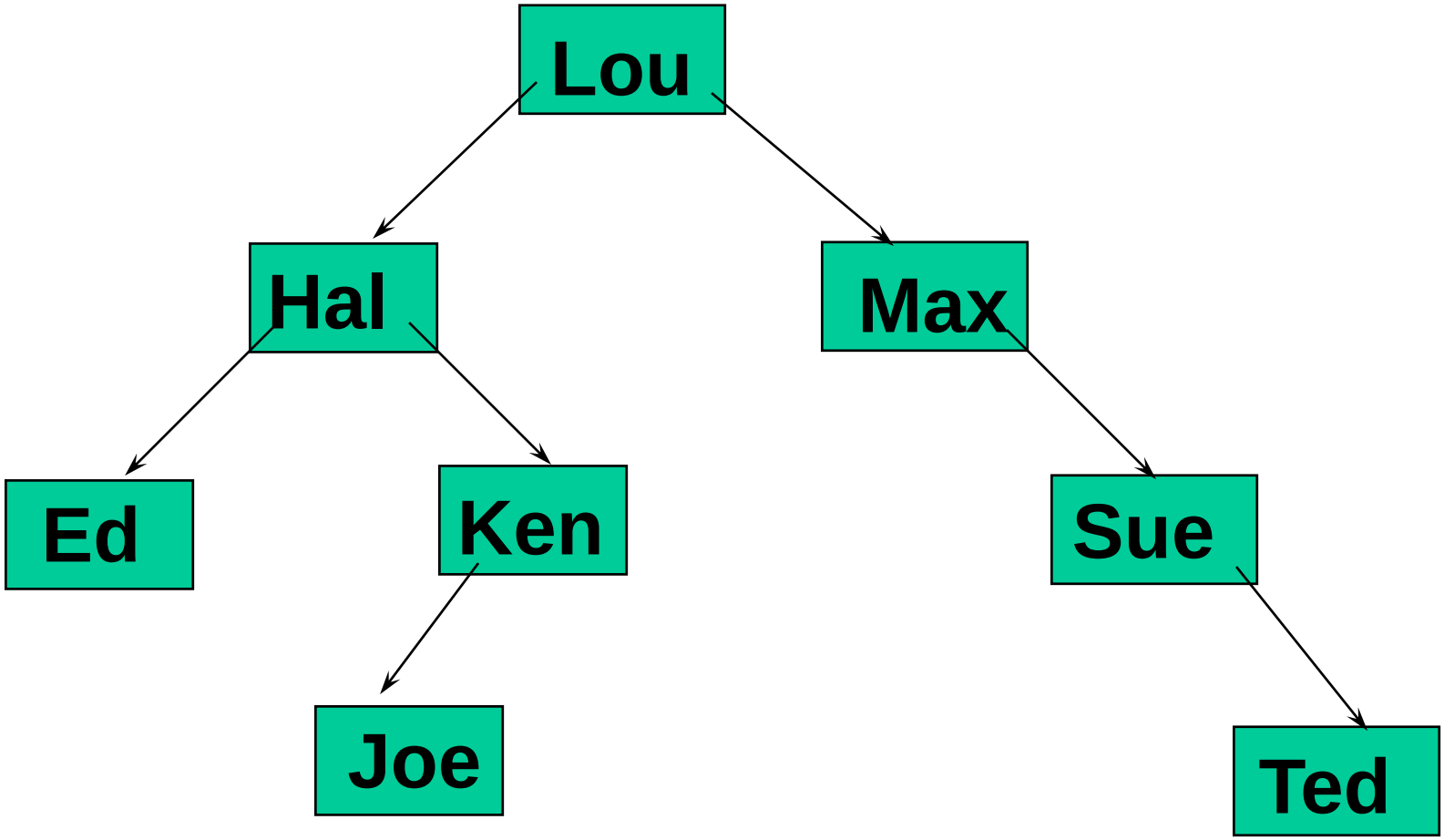
How many descendants of Hal?



Level

- The level of vertex v in a rooted tree is the length of the unique path from the root to v.

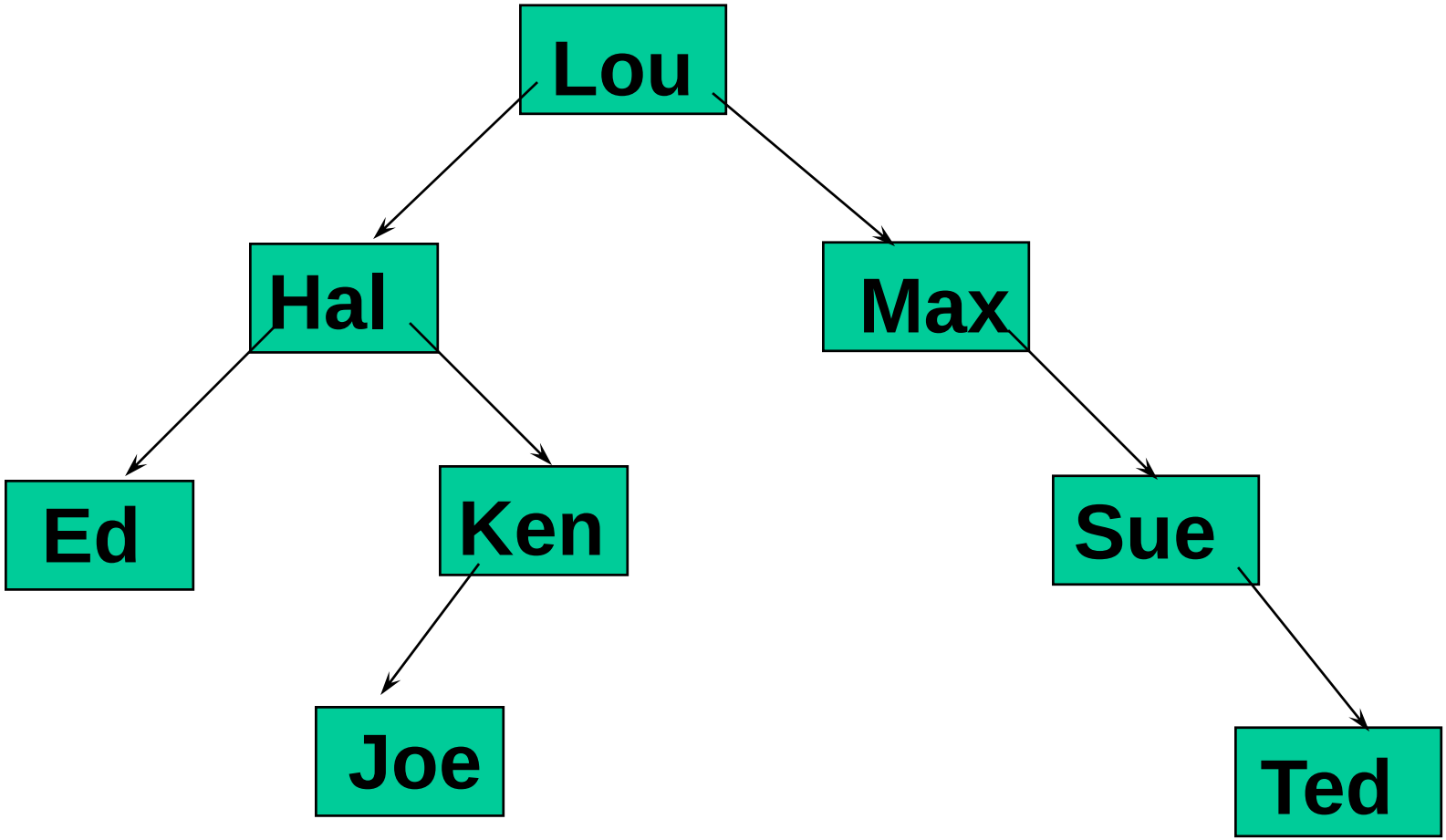
What is the level of Ted?



Height

- The height of a rooted tree is the maximum of the levels of its vertices.

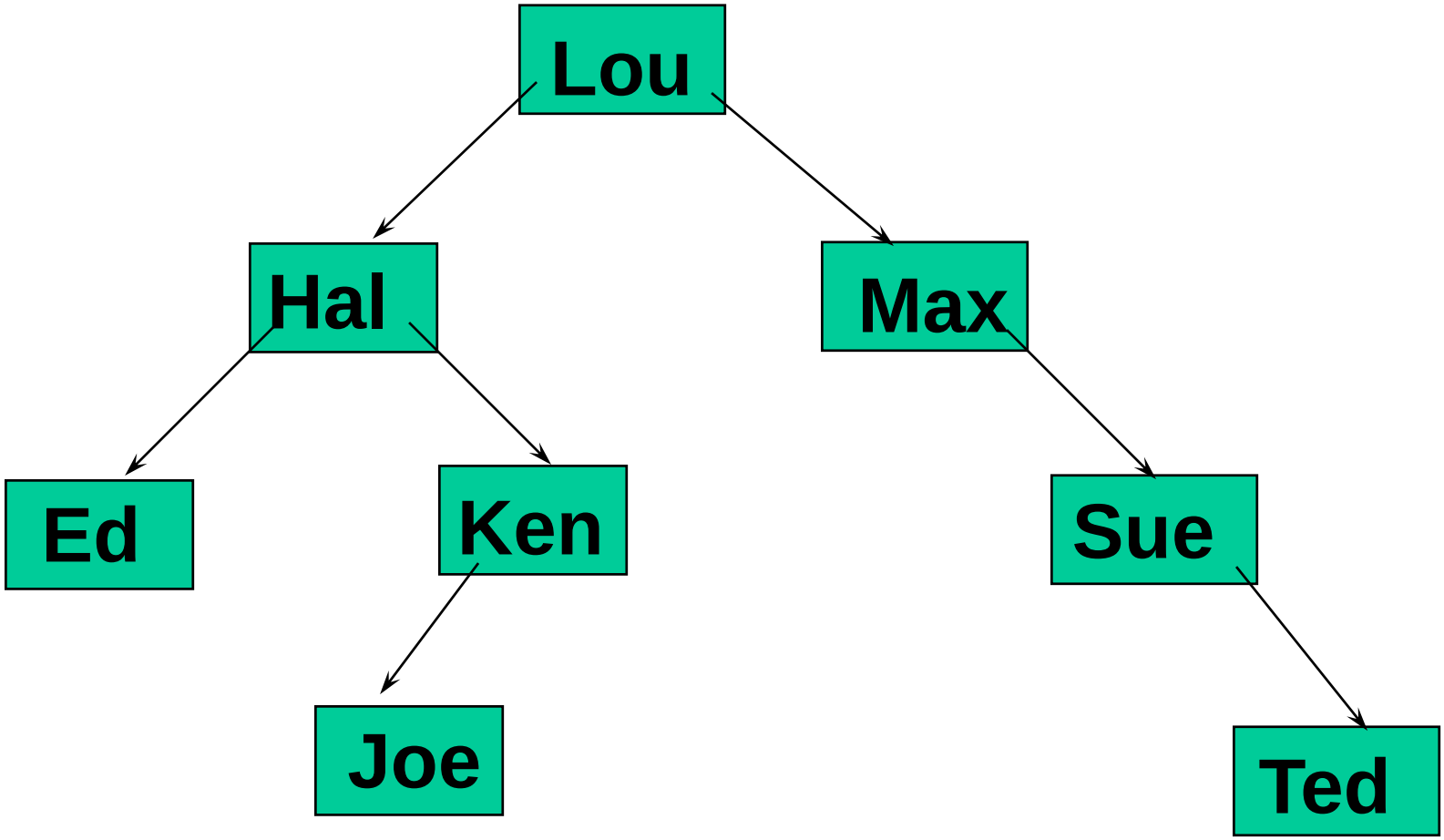
What is the height?



Balanced

- A rooted binary tree of height H is called balanced if all its leaves are at levels H or H-1.

Is this binary tree balanced?



Tree Properties

- Theorem 2: A tree with n vertices has n-1 edges.
- Proof: Choose the vertex r as the root of the tree. We set up a one-to-one correspondence between the edges and the vertices other than r by associating the terminal vertex of an edge to that edge. Since there are n-1 vertices other than r, there are n-1 edges in the tree.

- Theorem 3: A full m-ary tree with i internal vertices contains $n=mi+1$ vertices.

***** Definition: m-ary tree, full m-ary tree

Tree Properties

- **Theorem 4: A full *m-ary* tree with**
  - ***n* vertices has *i*=(*n*-1)/m internal vertices and *l*=[(*m*-1)*n*+1]/*m* leaves**
  - ***i* internal vertices has *n*=*mi*+1 vertices and *l*=(*m*-1)*i*+1 leaves**
  - ***l* leaves has *n*=(*ml*-1)/(*m*-1) vertices and *i*=(*l*-1)/(*m*-1) internal vertices**
- **Proof: very easy**

Tree Properties

- **Theorem 5.** **There are at most 2^h leaves in a binary tree of height h.**

Proof: mathematical induction

- **Corollary:** **If a binary tree with l leaves is full and balanced, then its height is**

$$h = \lceil \log_2 l \rceil .$$

*** This theorem can be extended to *m*-ary tree (p.790)**

Tree as Models

- Saturated Hydrocarbons and Trees
- Representing Organizations
- Computer File Systems
- Tree-connected Parallel Processors

Exercises

- **Page 791: 11-13, 19-20, 23, 26**

11.2 Applications of Trees

- How should items in a list be stored so that an item can be easily located?

  Binary search trees

- What series of decisions should be made to find an object with a certain property in a collection of objects of a certain type?

  Decision trees

- How should a set of characters be efficiently coded by bit strings?

  Prefix codes

Binary Search Trees

- The goal in computer programs is to find any stored item efficiently when all stored items are ordered.
- A Binary Search Tree can be used to store items in its vertices. It enables efficient searches.
- Binary search tree is a special kind of binary tree in which:
  - Each vertex contains a distinct key value,
  - The key values in the tree can be compared using “greater than” and “less than”, and
  - The key value of each vertex in the tree is less than every key value in its right subtree, and greater than every key value in its left subtree.

Shape of a binary search tree . . .

Depends on its key values and their order of insertion.

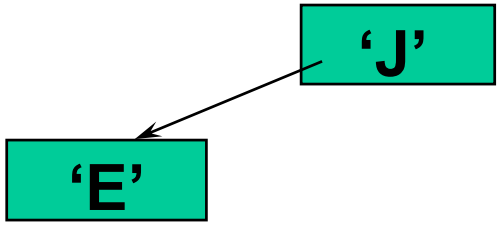
Insert the elements ‘J’ ‘E’ ‘F’ ‘T’ ‘A’ in that order.

The first value to be inserted is put into the root.



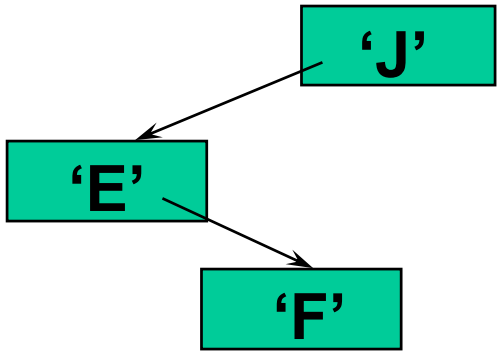
Inserting ‘E’ into the BST

Thereafter, each value to be inserted begins by comparing itself to the value in the root, moving left it is less, or moving right if it is greater. This continues at each level until it can be inserted as a new leaf.



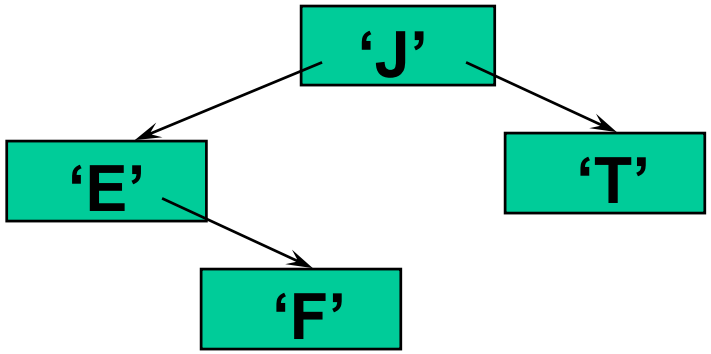
Inserting ‘F’ into the BST

Begin by comparing ‘F’ to the value in the root, moving left it is less, or moving right if it is greater. This continues until it can be inserted as a leaf.



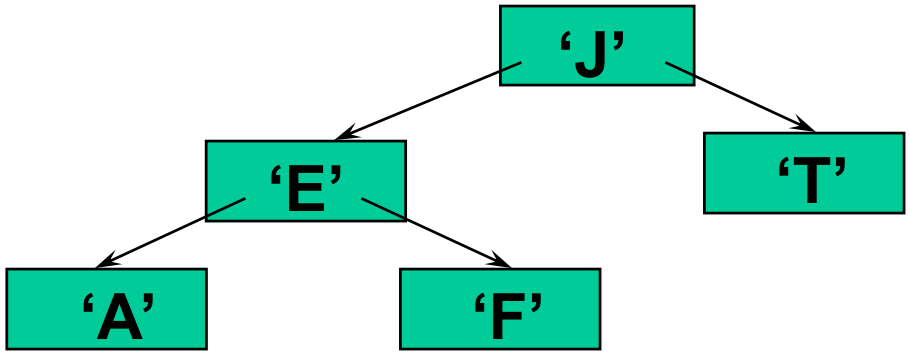
Inserting ‘T’ into the BST

Begin by comparing ‘T’ to the value in the root, moving left it is less, or moving right if it is greater. This continues until it can be inserted as a leaf.



Inserting ‘A’ into the BST

Begin by comparing ‘A’ to the value in the root, moving left it is less, or moving right if it is greater. This continues until it can be inserted as a leaf.



What binary search tree . . .

is obtained by inserting

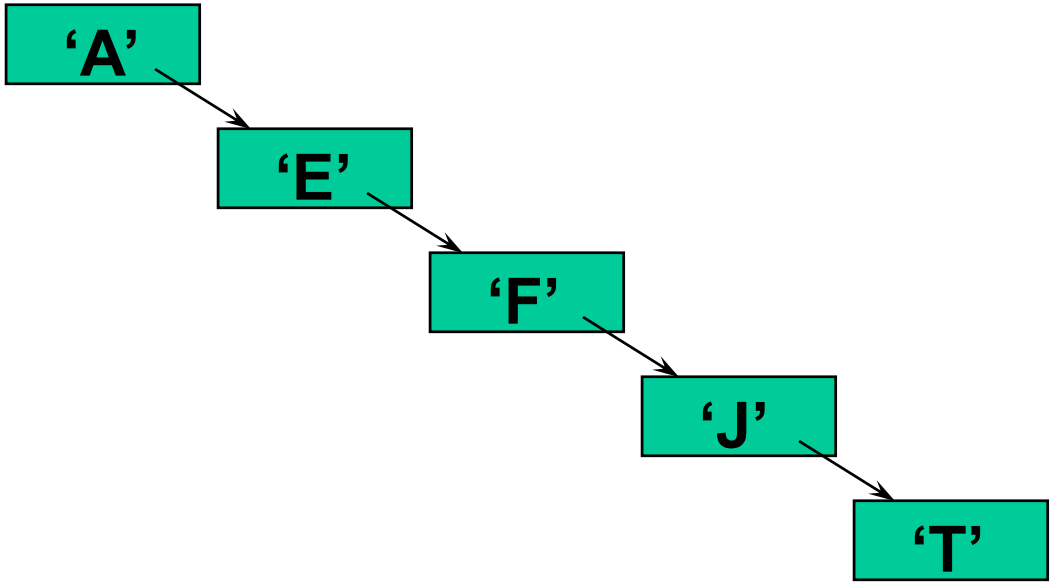
the elements ‘A’ ‘E’ ‘F’ ‘J’ ‘T’ in that order?

‘A’

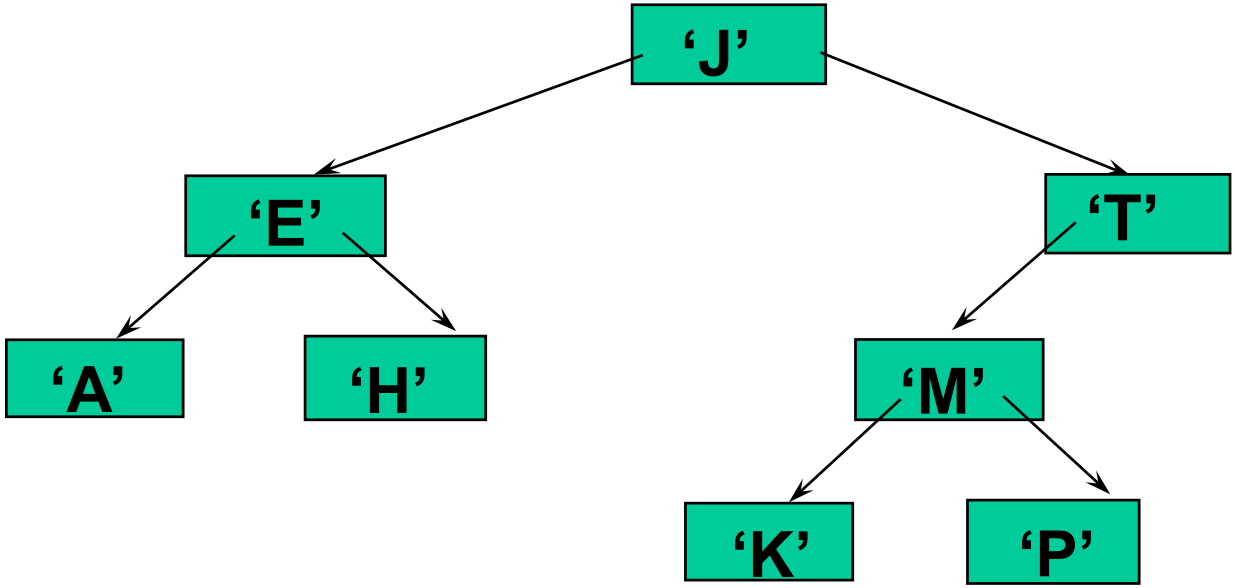
Binary search tree . . .

obtained by inserting

the elements ‘A’ ‘E’ ‘F’ ‘J’ ‘T’ in that order.



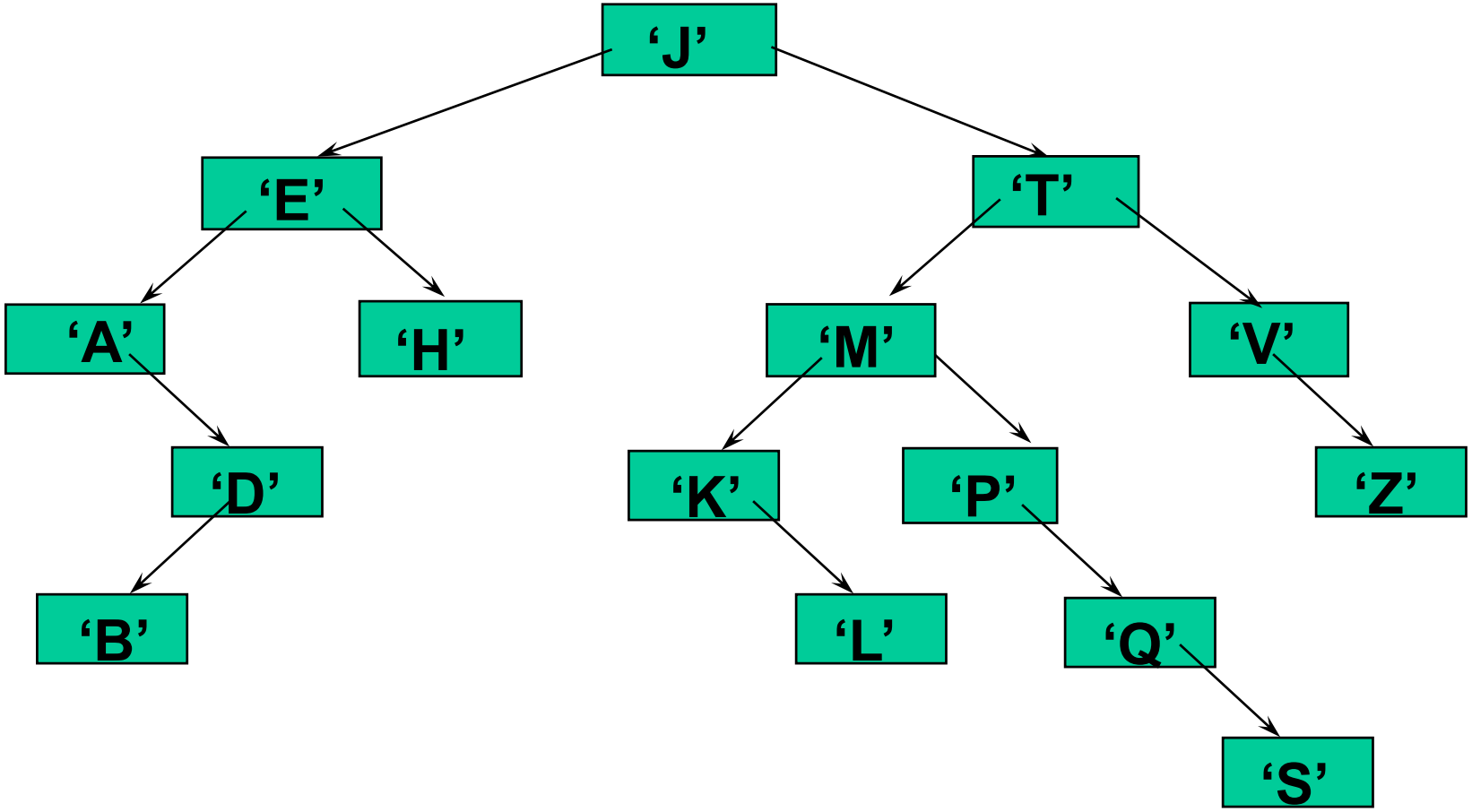
Another binary search tree



Add nodes containing these values in this order:

‘D’ ‘B’ ‘L’ ‘Q’ ‘S’ ‘V’ ‘Z’

Is ‘F’ in the binary search tree?



Binary Search Tree Algorithm

- Page 795
- Balanced binary search trees give optimal worst-case complexity for binary searching.

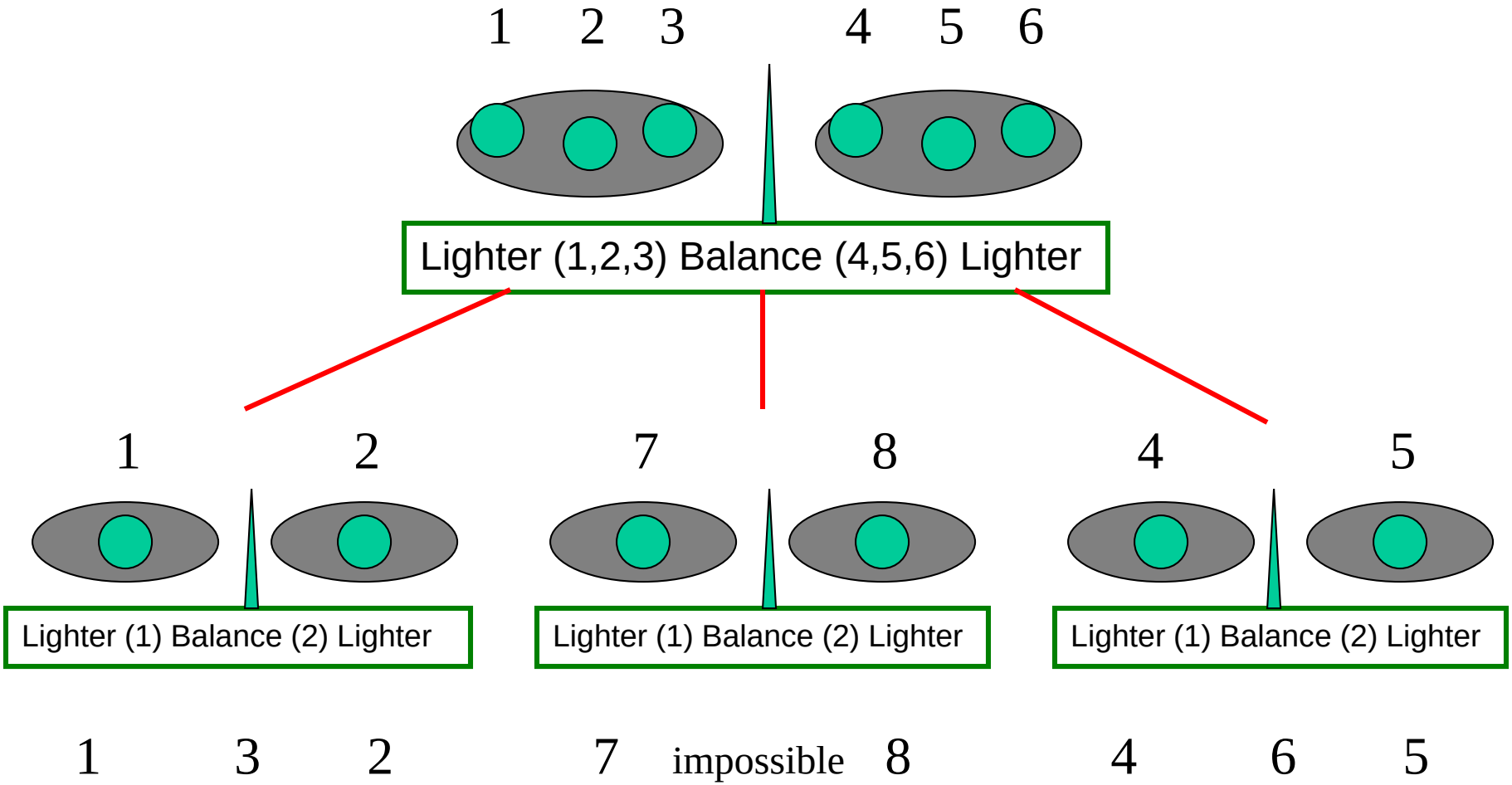
  Why ?

Decision Trees

- Rooted trees can be used to model problems in which a series of decisions leads to a solution.

- A rooted tree in which each internal vertex corresponds to a decision, with a subtree at these vertices for each possible outcome of the decision, is called a decision tree.

Counterfeit coin detection



Binary Comparison based Sorting

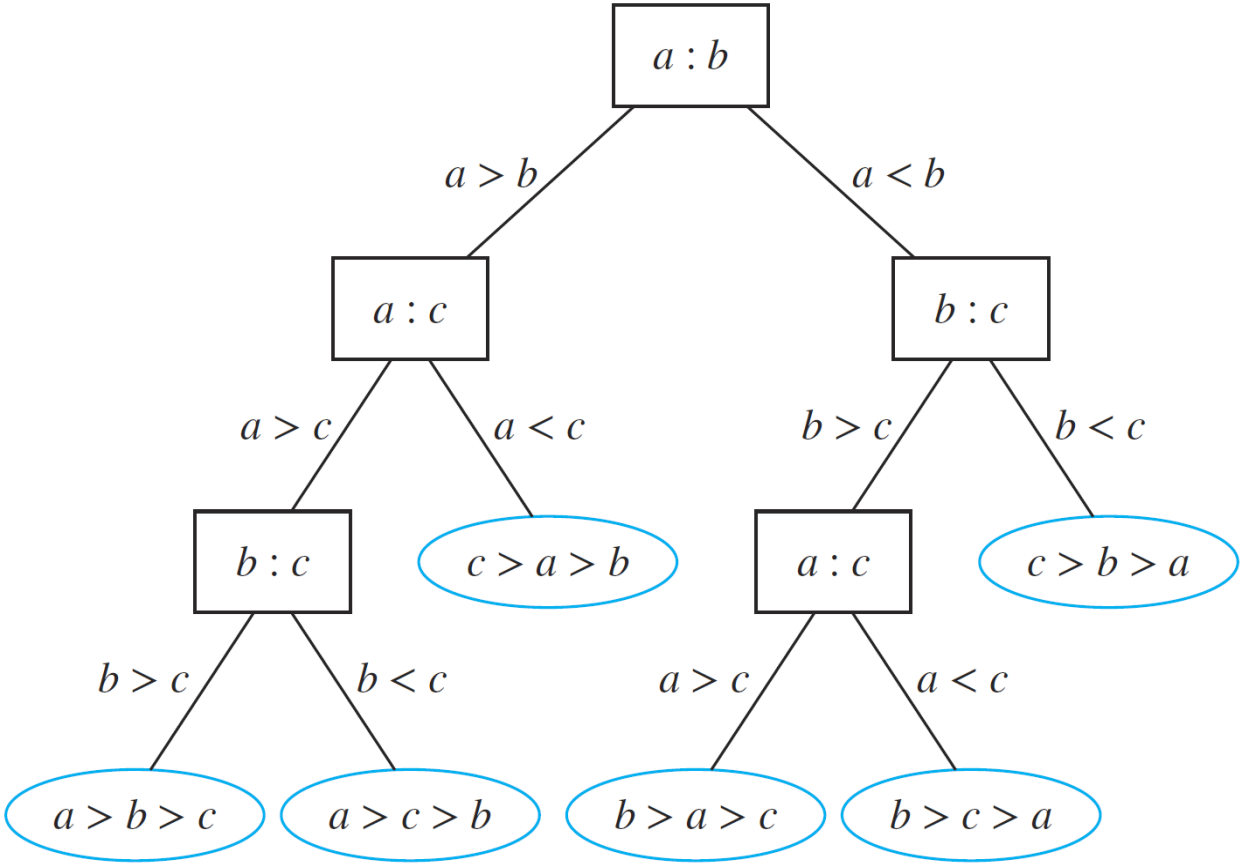


FIGURE 4 **A decision tree for sorting three distinct elements.**

Binary Comparison based Sorting

- **Theorem 1: A sorting algorithm based on binary comparisons requires at least $\lceil \log_2 n! \rceil$ comparisons.**
- **Corollary 1: The number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log_2 n)$.**
- **Theorem 2: The average number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log_2 n)$.**

Prefix Codes

- Using bit strings of different lengths to encode letters can improve coding efficiency.

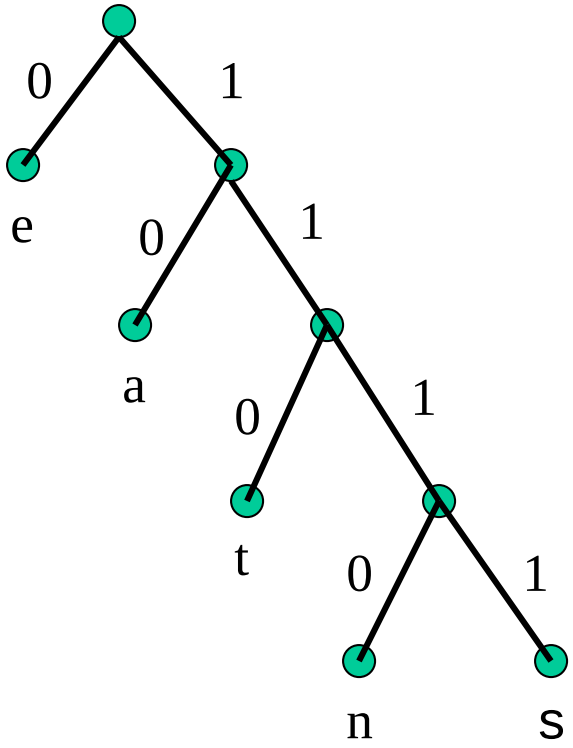
- When letters are encoded using varying numbers of bits, some method must be used to determine where the bits for each character start and end.

  e: 0 a: 1 t: 01

  0101: eat, tea, eaea, tt

- To ensure that no bit string corresponds to more than one sequence of letters, the bit string for a letter must never occur as the first part of the bit string for another letter. Codes with this property are called prefix codes.

Prefix code VS. Binary tree



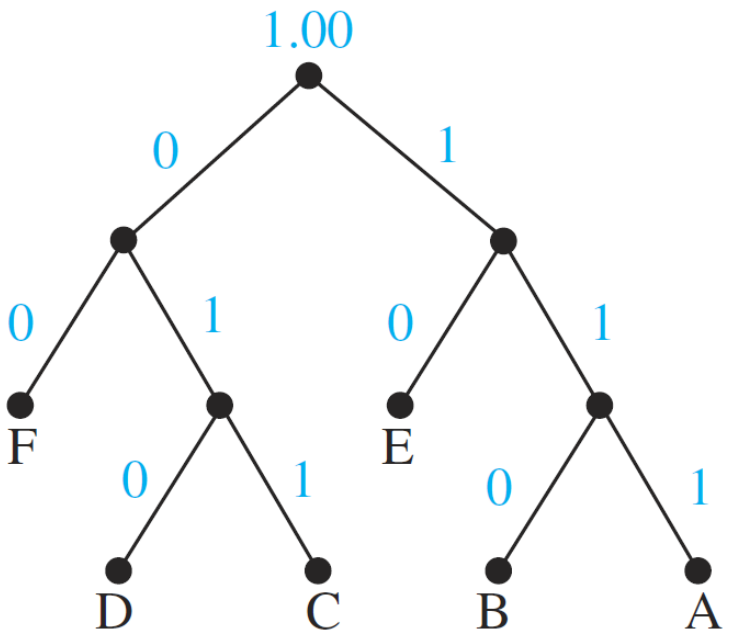
Huffman Coding

ALGORITHM 2 Huffman Coding.

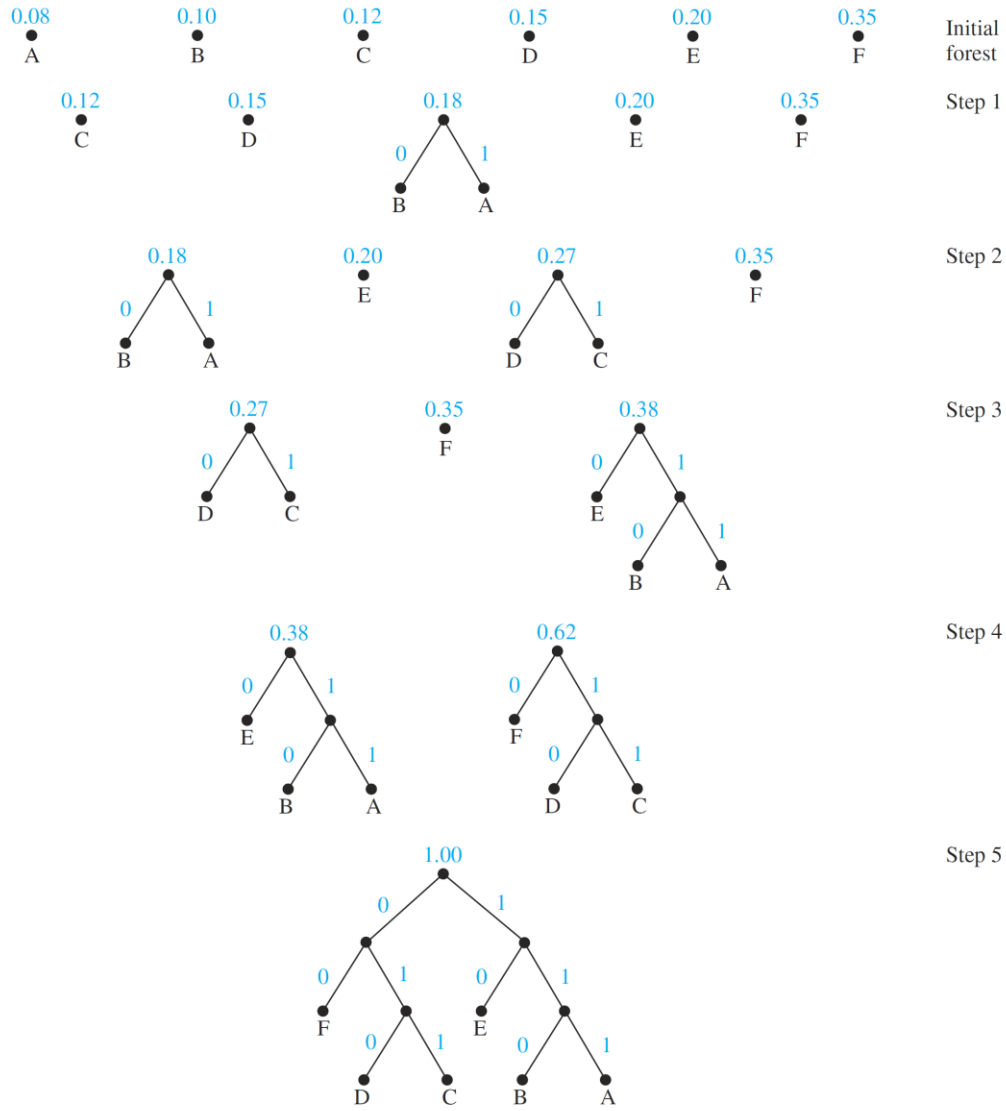
procedure *Huffman*(C: symbols a_i with frequencies w_i, $i = 1, \ldots, n$)
F := forest of n rooted trees, each consisting of the single vertex a_i and assigned weight w_i
while F is not a tree
  Replace the rooted trees T and T' of least weights from F with $w(T) \geq w(T')$ with a tree having a new root that has T as its left subtree and T' as its right subtree. Label the new edge to T with 0 and the new edge to T' with 1.
  Assign $w(T) + w(T')$ as the weight of the new tree.
{the Huffman coding for the symbol a_i is the concatenation of the labels of the edges in the unique path from the root to the vertex a_i }

Huffman Coding

- Use Huffman coding to encode the following symbols with the frequencies listed: A: 0.08, B: 0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35. What is the average number of bits used to encode a character?



Huffman Coding



*Game Trees

- **Self-study**

Exercises

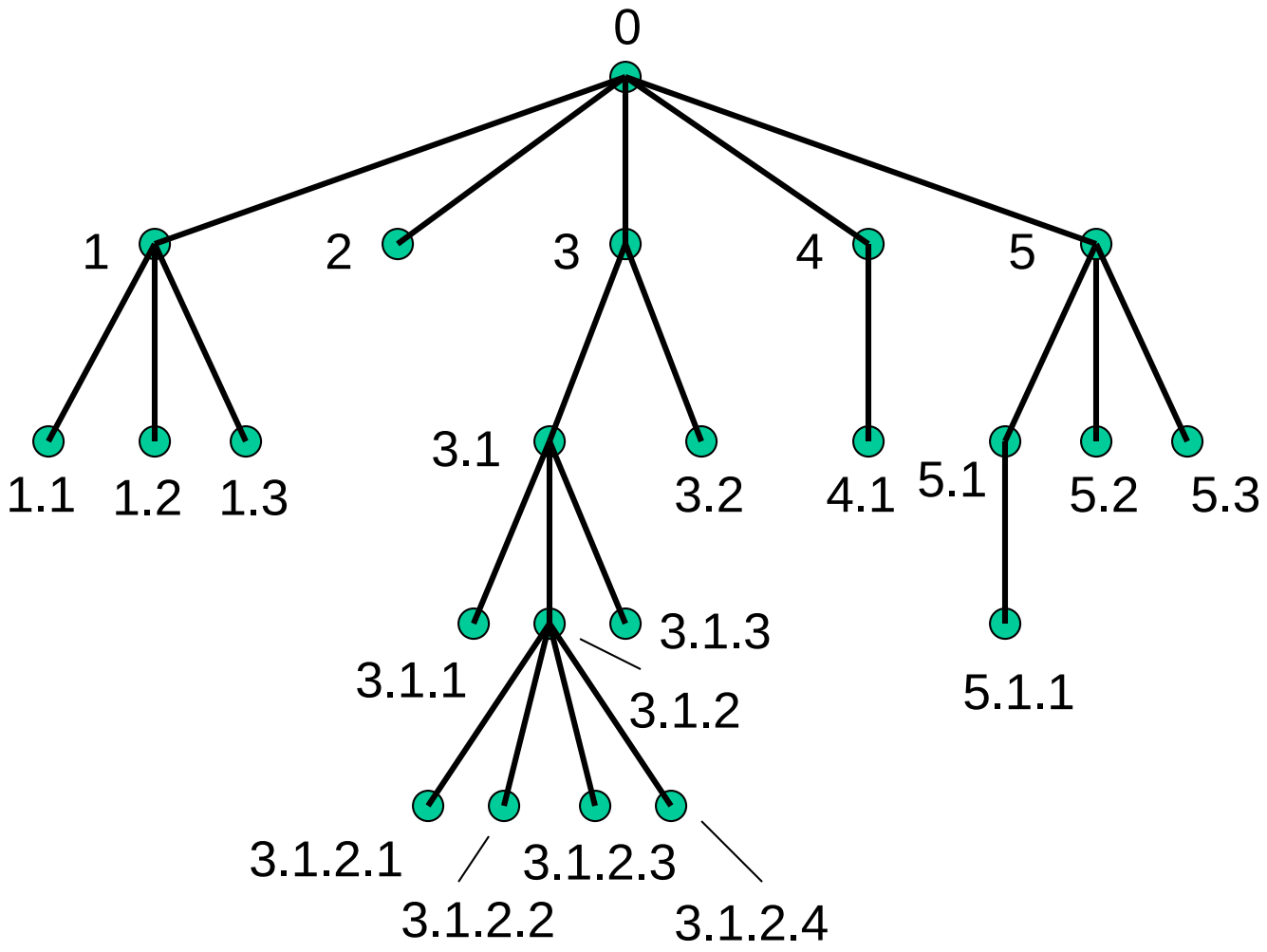
- **Page 805: 5, 9, 11, 21, 24**

11.3 Tree Traversal

- Visiting all the vertices of an ordered rooted tree.

- Universal address system: Procedures for traversing all vertices of an ordered rooted tree rely on the orderings of children. Universal address system is one way among them with:
  - Label the root with the integer 0. Then label its k children (at level 1) from left to right with 1,2,3,…,k.
  - For each vertex v at level n with label A, label its k_v children, as they are drawn from left to right, with $A.1, A.2, \ldots, A.k_v$.

The universal address system of an ordered rooted tree



Traversal Algorithms

- A traversal algorithm is a procedure for systematically visiting every vertex of an ordered binary tree.
- Tree traversals are defined recursively.
- Three traversals are named:

  preorder,

  inorder,

  postorder.

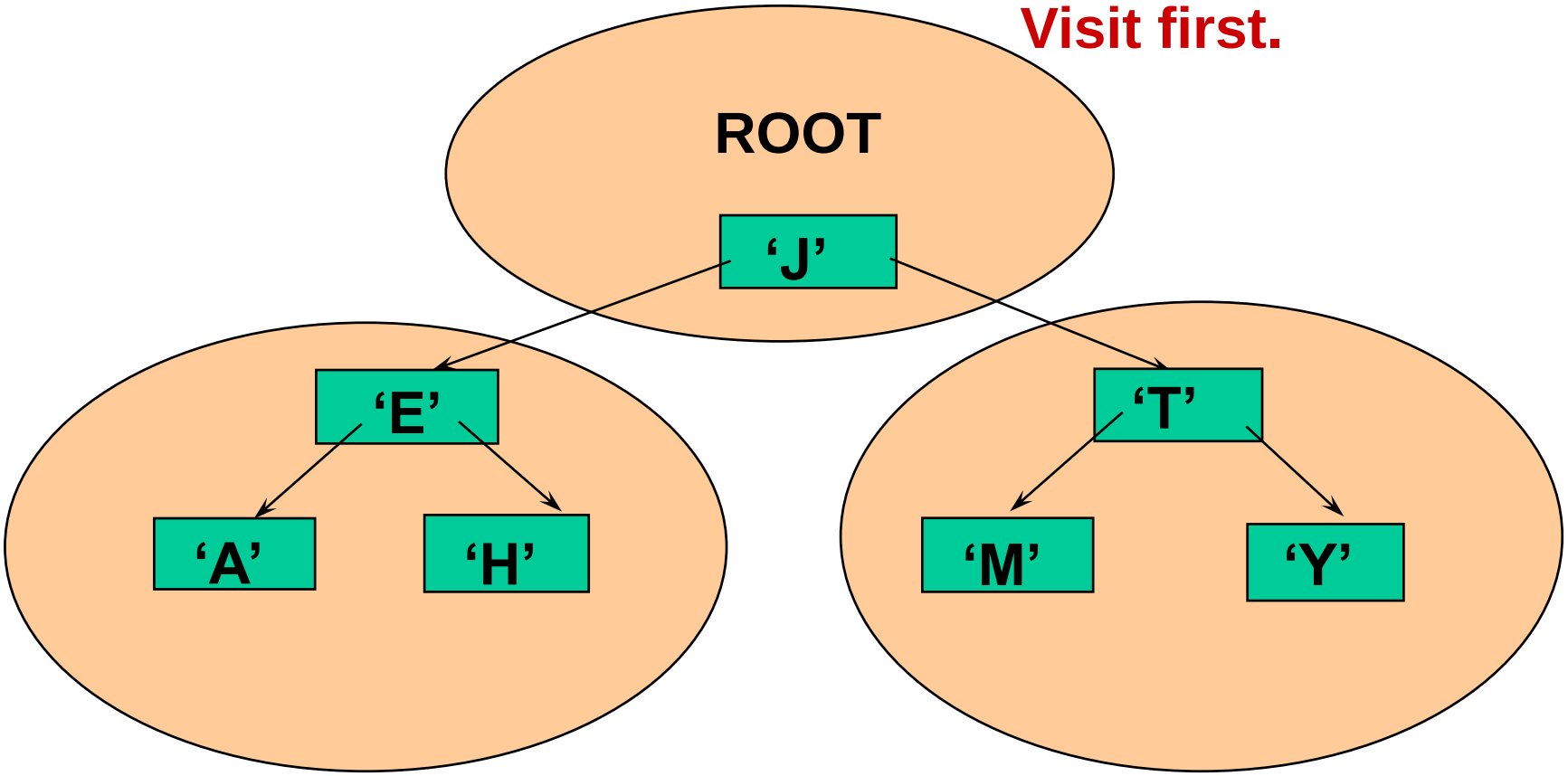
PREORDER Traversal Algorithm

Let T be an ordered rooted tree with root r.

If T has only r, then r is the preorder traversal.

Otherwise, suppose T_1, T_2 , …, T_n are the left to right subtrees at r. The preorder traversal begins by visiting r. Then traverses T_1 in preorder, then traverses T_2 in preorder, and so on, until T_n is traversed in preorder.

Preorder Traversal: J E A H T M Y



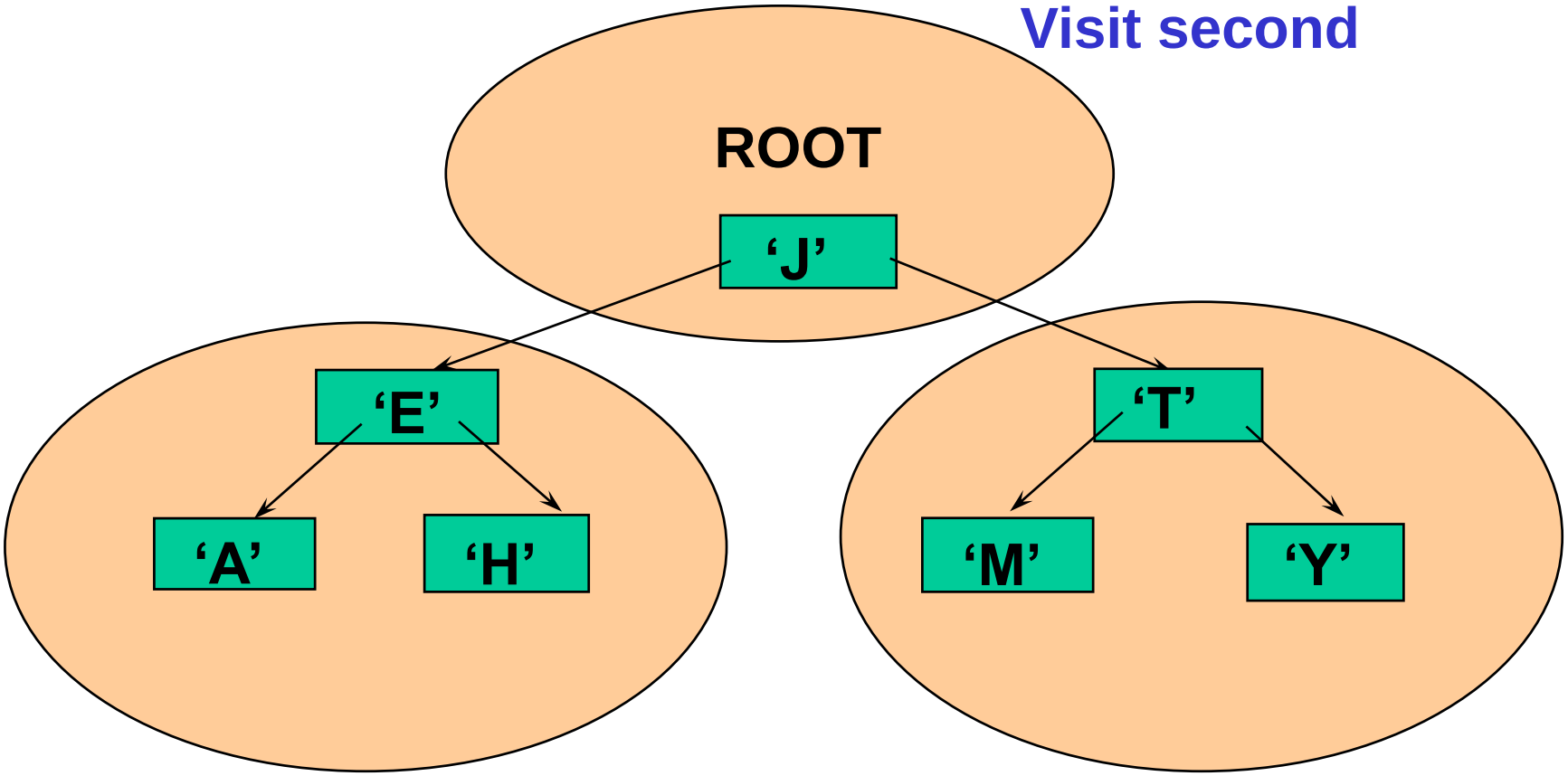
INORDER Traversal Algorithm

Let T be an ordered rooted tree with root r.

If T has only r, then r is the inorder traversal.

Otherwise, suppose T_1, T_2,…, T_n are the left to right subtrees at r. The inorder traversal begins by traversing T_1 in inorder. Then visits r, then traverses T_2 in inorder, and so on, until T_n is traversed in inorder.

Inorder Traversal: A E H J M T Y



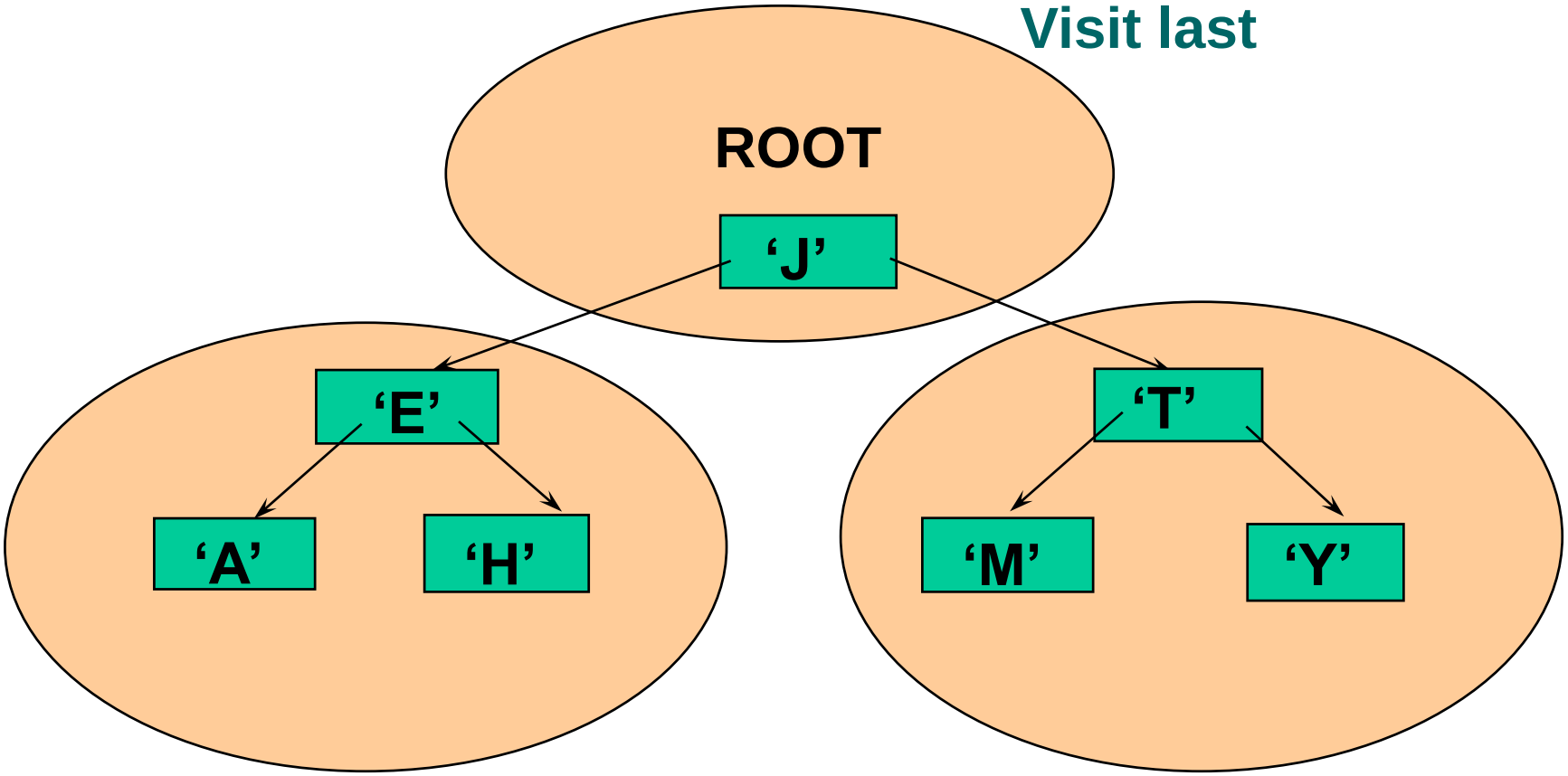
POSTORDER Traversal Algorithm

Let T be an ordered rooted tree with root r.

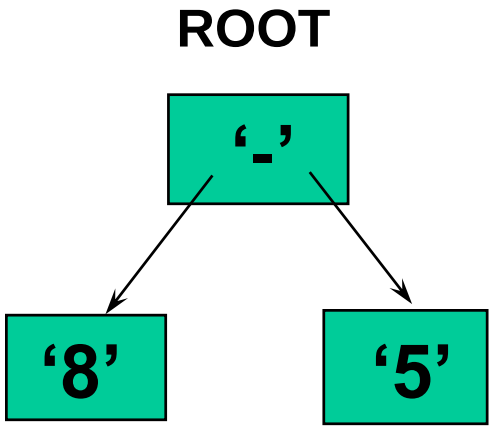
If T has only r, then r is the postorder traversal.

Otherwise, suppose T_1, T_2,…, T_n are the left to right subtrees at r. The postorder traversal begins by traversing T_1 in postorder. Then traverses T_2 in postorder, until T_n is traversed in postorder, finally ends by visiting r.

Postorder Traversal: A H E M Y T J



A Binary Expression Tree



INORDER TRAVERSAL (INFIX): `8 - 5 has value 3`

PREORDER TRAVERSAL(PREFIX): `- 8 5`

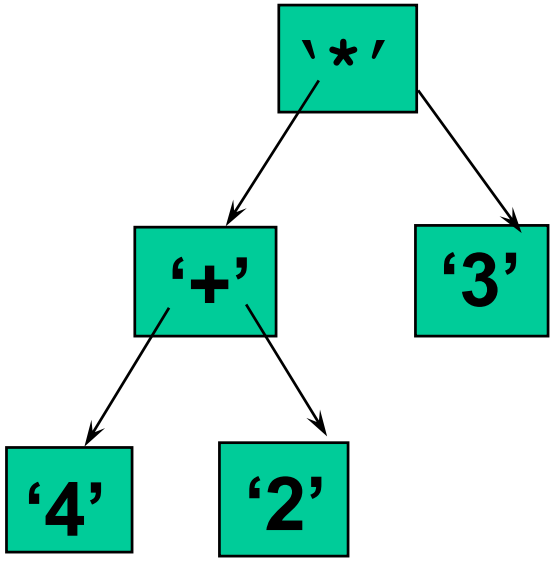
POSTORDER TRAVERSAL(POSTFIX): `8 5 -`

A Binary Expression Tree is . . .

A special kind of binary tree in which:

1. Each leaf node contains a single operand,
2. Each nonleaf node contains a single binary operator, and
3. The left and right subtrees of an operator node represent subexpressions that must be evaluated before applying the operator at the root of the subtree.

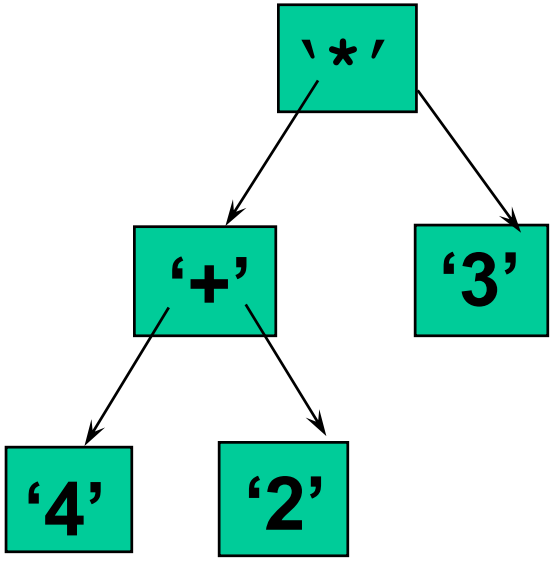
A Binary Expression Tree



What value does it have?

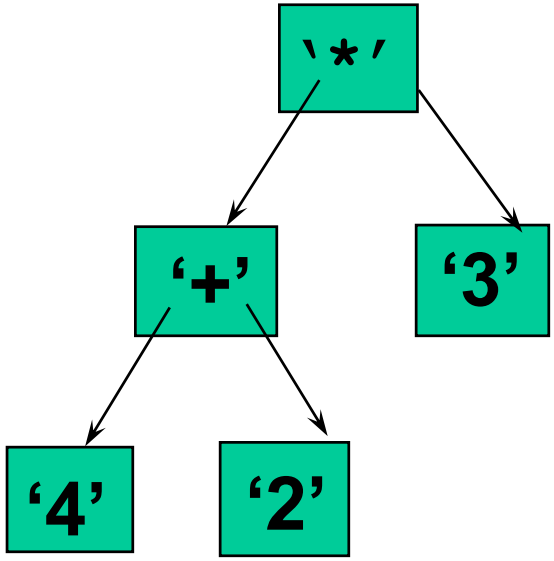
(4 + 2) * 3 = 18

A Binary Expression Tree



What infix, prefix, postfix expressions does it represent?

A Binary Expression Tree



Infix: ((4 + 2) * 3)

Prefix: * + 4 2 3 *evaluate from right*

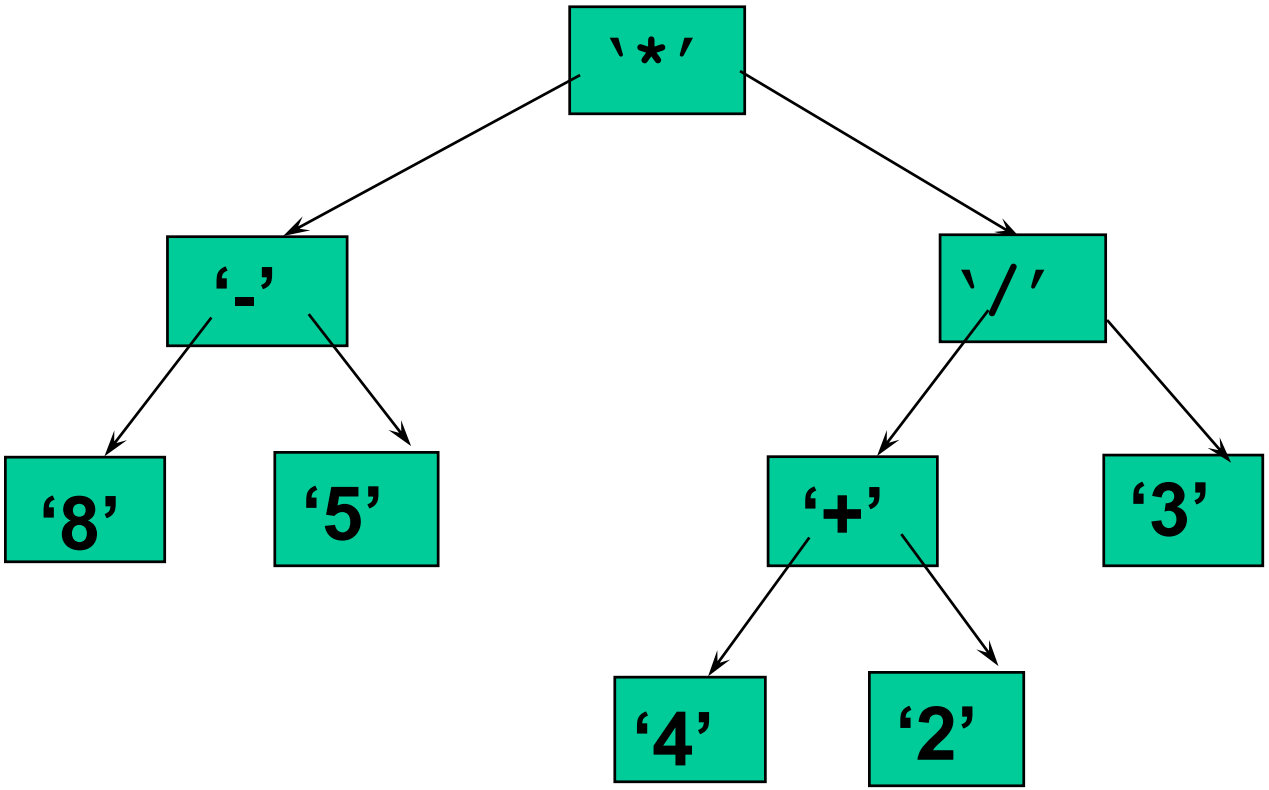
Postfix: 4 2 + 3 * *evaluate from left*

Levels Indicate Precedence

When a binary expression tree is used to represent an expression, the levels of the nodes in the tree indicate their relative precedence of evaluation.

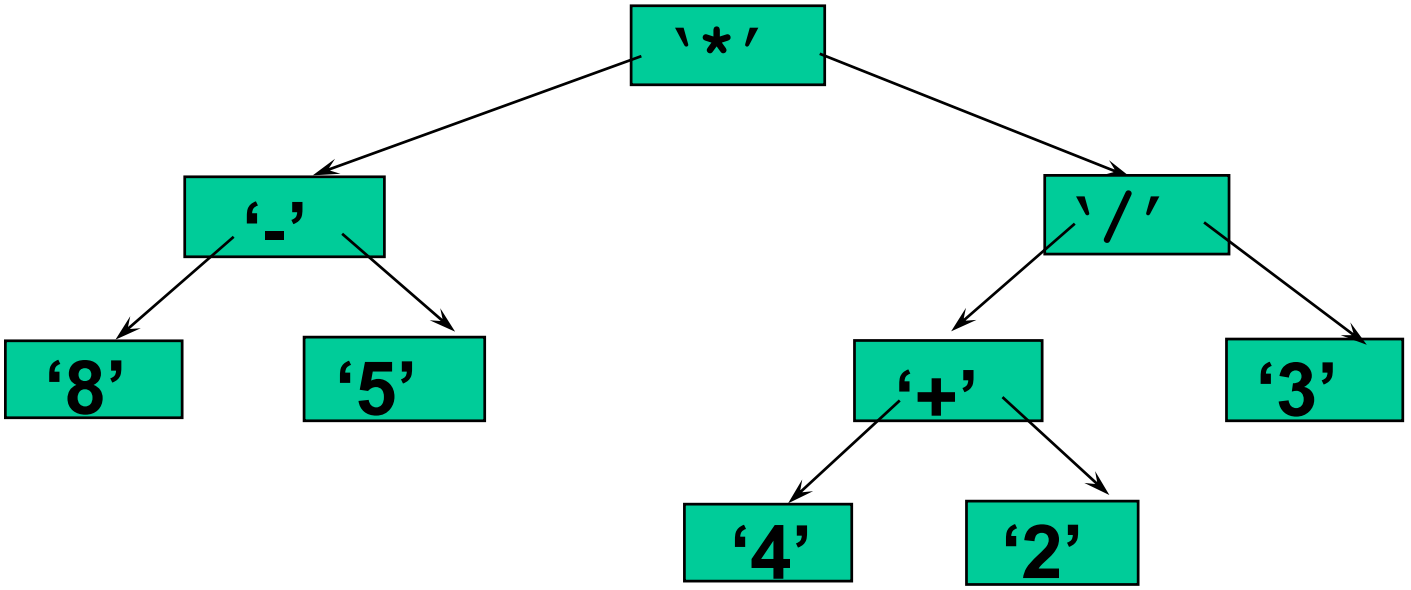
Operations at higher levels of the tree are evaluated later than those below them. The operation at the root is always the last operation performed.

Evaluate this binary expression tree



What infix, prefix, postfix expressions does it represent?

A binary expression tree

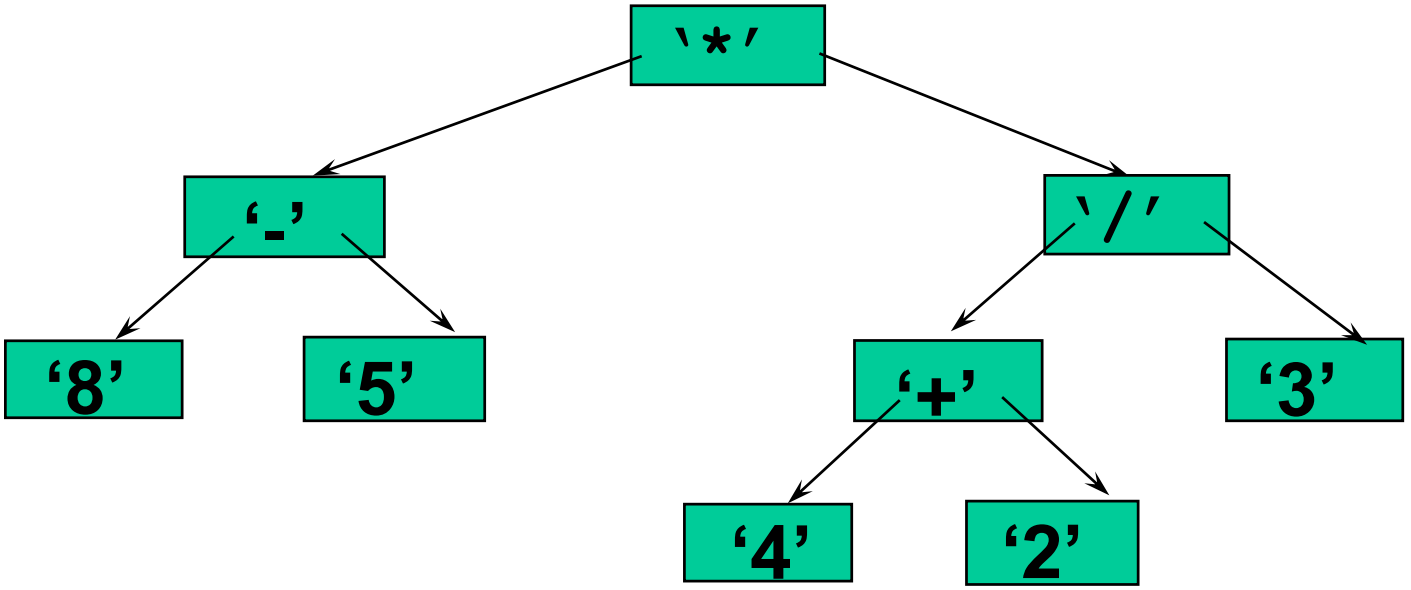


Infix: **((8 - 5) * ((4 + 2) / 3))**

Prefix: *** - 8 5 / + 4 2 3**

Postfix: **8 5 - 4 2 + 3 / *** ***has operators in order used***

A binary expression tree



Infix: **((8 - 5) * ((4 + 2) / 3))**

Prefix: *** - 8 5 / + 4 2 3** ***evaluate from right***

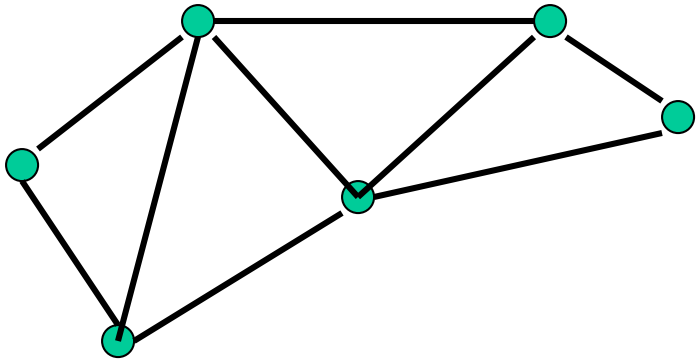
Postfix: **8 5 - 4 2 + 3 / *** ***evaluate from left***

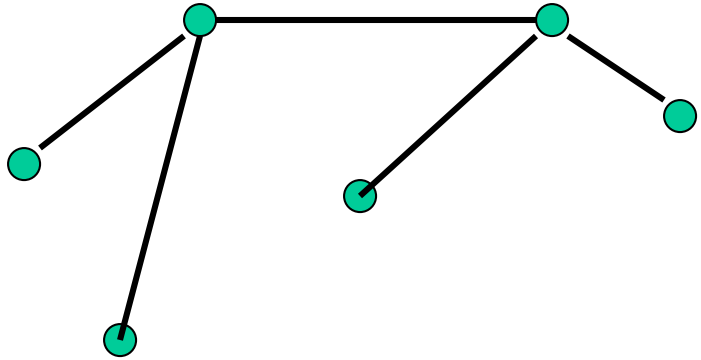
Exercises

- **Page 819: 9, 12, 15, 19, 21, 23, 24**

11.4 Spanning Trees

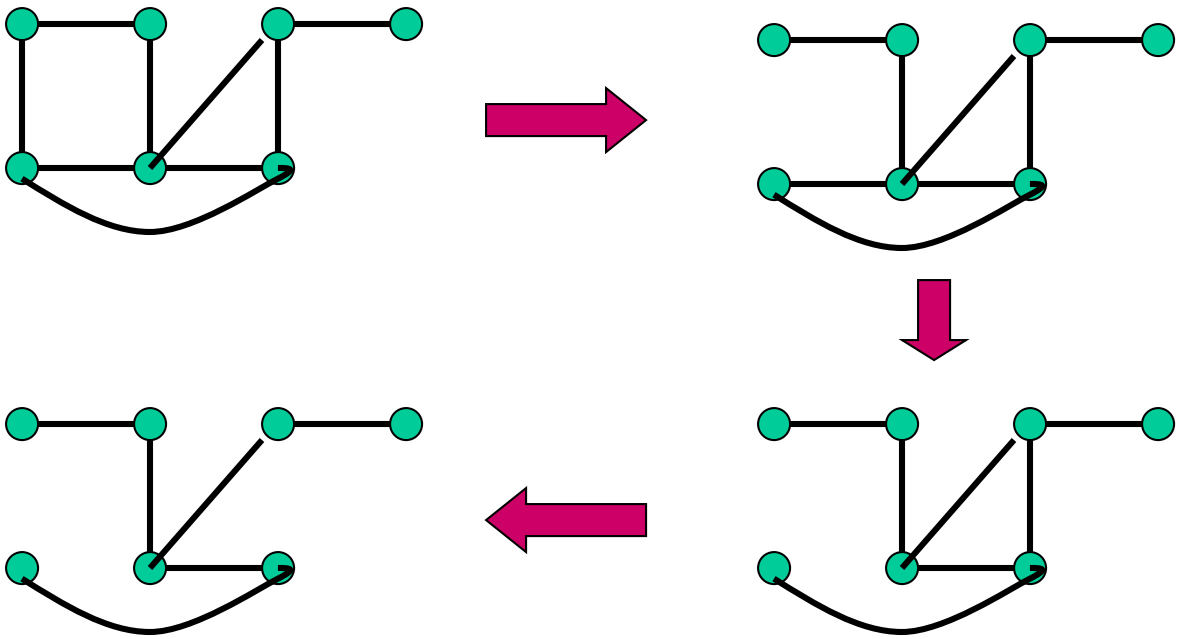
- Problem: Find a connected subgraph with the minimum number of edges containing all vertices of the original simple graph.



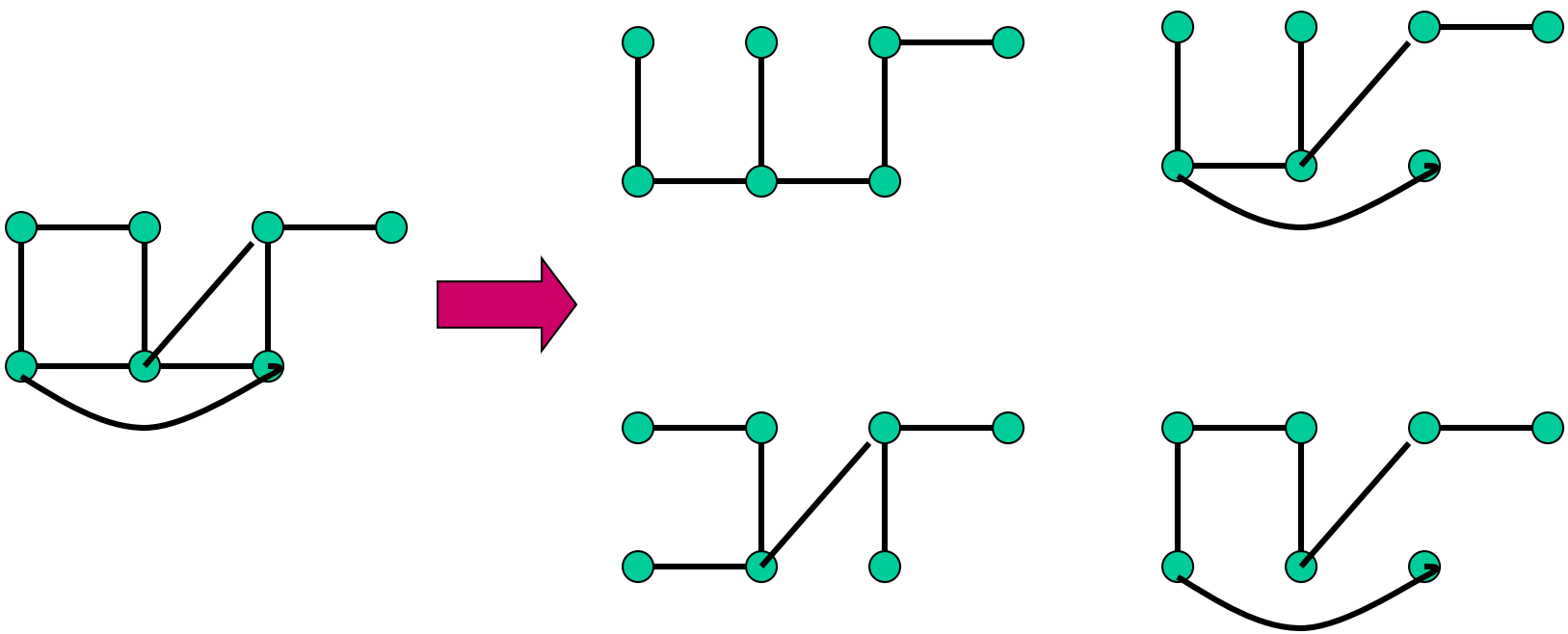


Spanning Tree

- Definition 1: Let *G* be a simple graph. A spanning tree of *G* is a subgraph of *G* that is a tree containing every vertex of *G*.



More than one spanning tree for a simple graph



Theorem

- Theorem 1: A simple graph is connected if and only if it has a spanning tree.
- Proof: First, suppose that a simple graph *G* has a spanning tree *T*. *T* contains every vertex of *G*. There is a path in *T* between any two of its vertices. Since *T* is a subgraph of *G*, there is a path in *G* between any two of its vertices. Hence *G* is connected.

  Second, suppose that *G* is connected. If *G* is not a tree, it must contain a simple circuit. Remove an edge from one of these simple circuits. The resulting subgraph has one fewer edge but still contains all vertices of *G* and is connected. If this subgraph is not a tree, it has a simple circuit; so as before, remove an edge that is in a simple circuit. Repeat this process until no simple circuits remain. The process terminates when no simple circuit remain. This tree is a spanning tree since it contains every vertex of *G*.

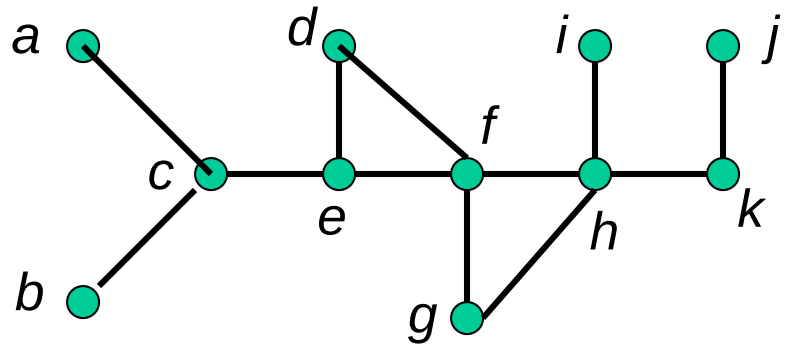
Algorithms for constructing spanning trees

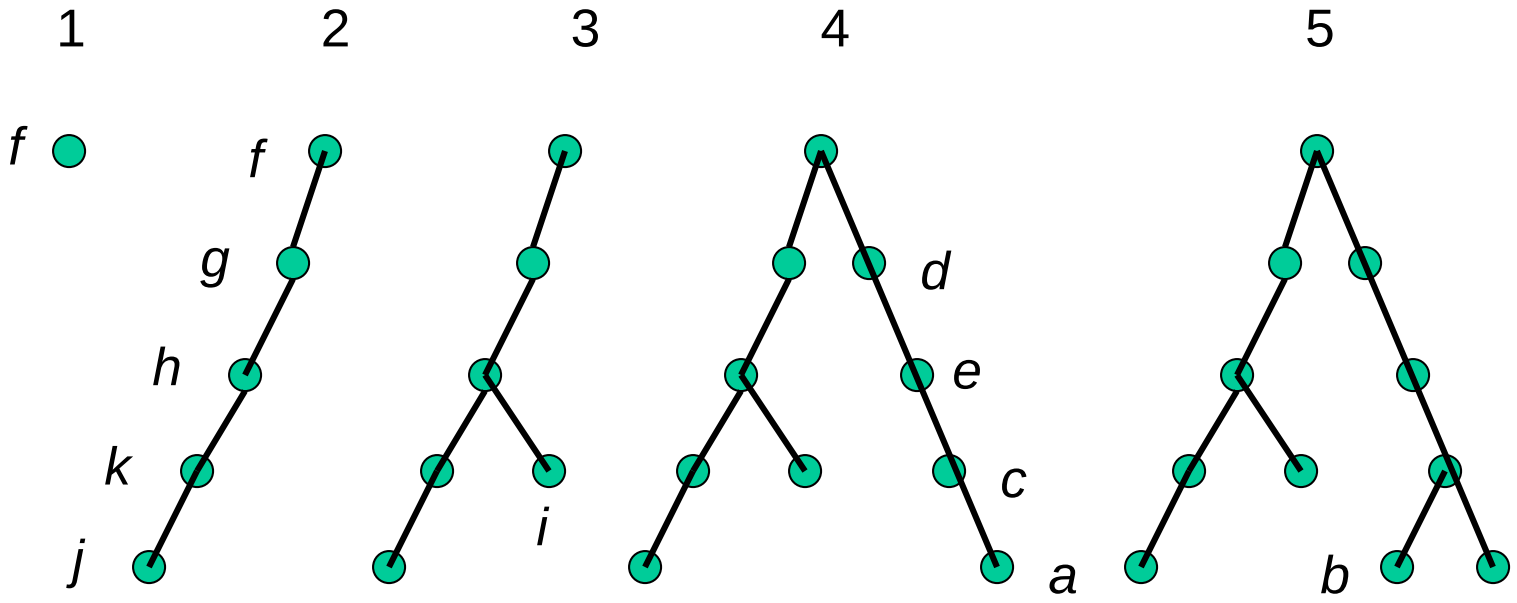
- Theorem 1 gives an algorithm for finding spanning trees by removing edges from simple circuits. This algorithm is inefficient, since it requires that simple circuits be identified.
- Instead of constructing spanning trees by removing edges, spanning trees can be built up by successively adding edges.
- Depth-first search and Breadth-first search

Depth-first search

- Arbitrarily choose a vertex of the graph as root.
- From a path starting at this vertex by successively adding edges, where each new edge is incident with the last vertex in the path and a vertex not already in the path.
- Continue adding edges to this path as long as possible.
- If the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree.
- If the path does not go through all vertices, more edges must be added. Move back to the next to last vertex in the path, if possible, form a new path starting at this vertex passing through vertices that were not already visited. If this cannot be done, move back another vertex in the path. Repeat this process.
- Depth-first search is also called backtracking.

Example

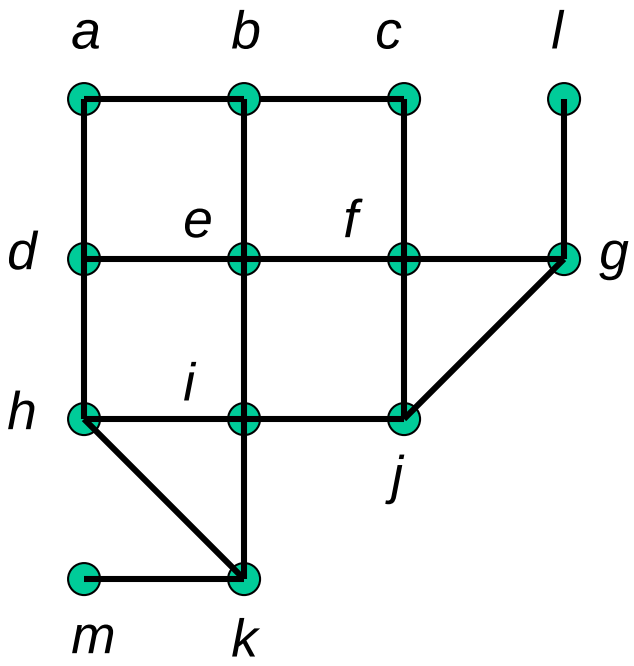


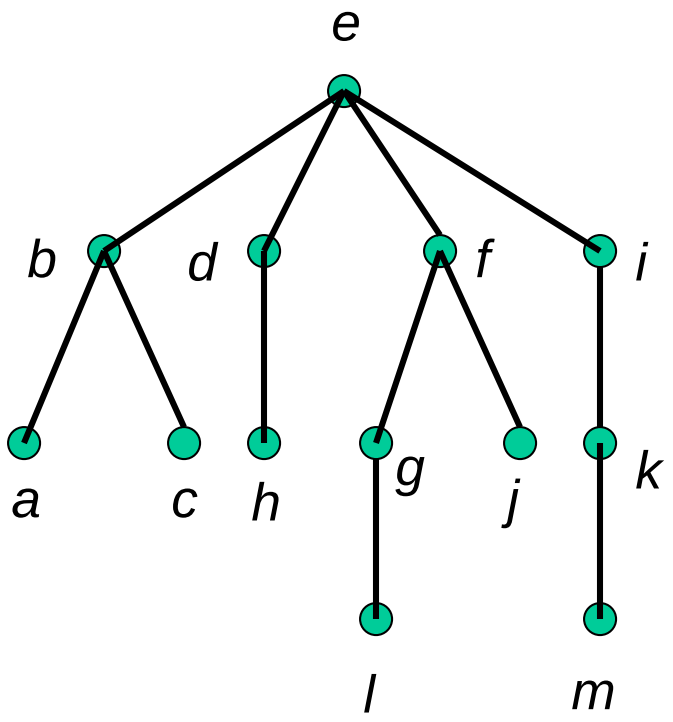


Breadth-first search

- Arbitrarily choose a root from the vertices of the graph.
- Add all edges incident to this vertex. The new vertices added at this stage become the vertices at level 1 in the spanning tree.
- For each vertex at level 1, visited in order, add each edge incident to this vertex to the tree as long as it does not produce a simple circuit.
- Arbitrarily order the children of each vertex at level 1. This produces the vertices at level 2 in the tree.
- Following the same procedure until all vertices in the tree have been added.

Example

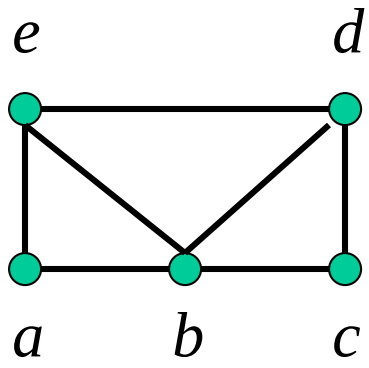




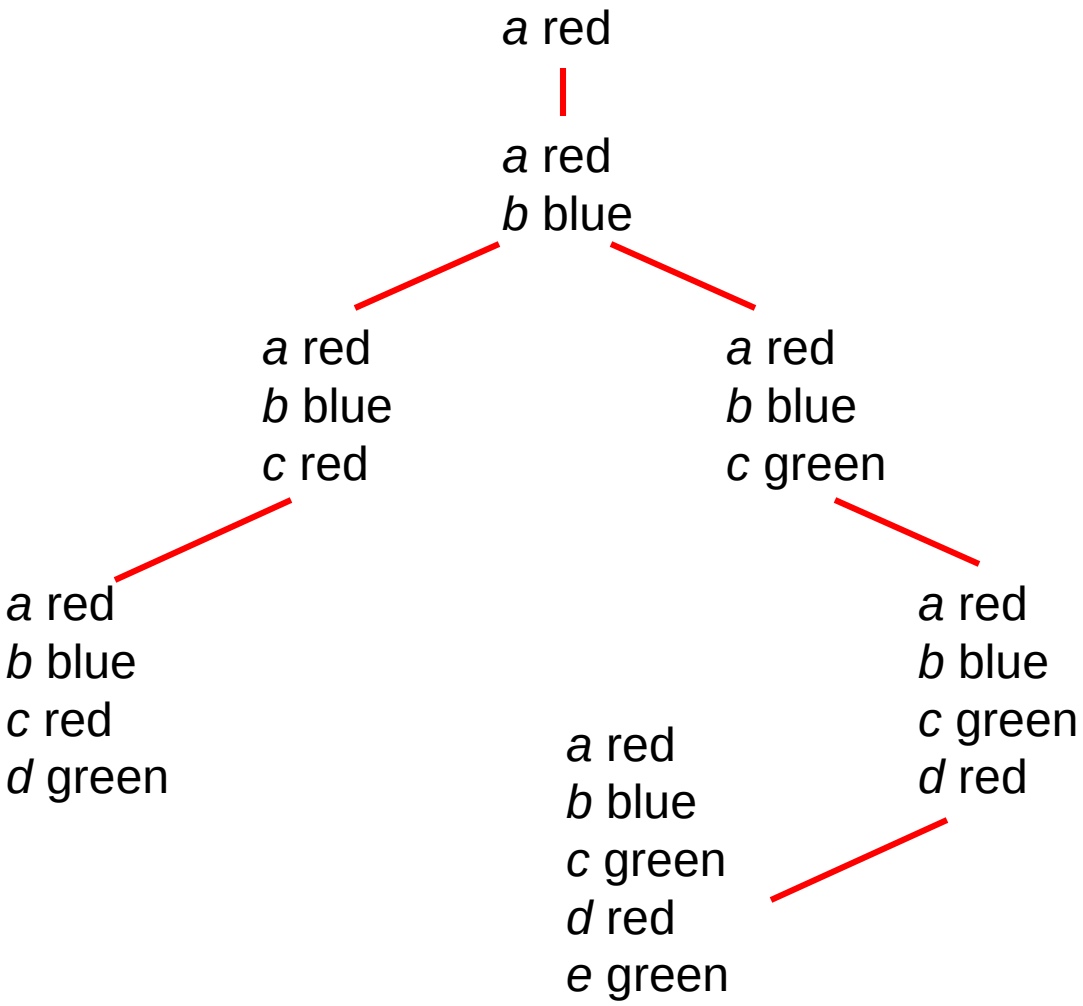
Backtracking Applications

- There are problems that can be solved only by performing an exhaustive search of all possible solutions. One way to search systematically for a solution is to use a decision tree, where each internal vertex represents a decision and each leaf a possible solution.
- To find a solution via backtracking, first make a sequence of decisions in an attempt to reach a solution as long as this is possible. The sequence of decisions can be represented by a path in the decision. Once it is known that no solution can result from any further sequence of decisions, backtrack to the parent of the current vertex and work toward a solution with another series of decisions, if this is possible. The procedure continues until a solution is found, or no solution existed.

Example

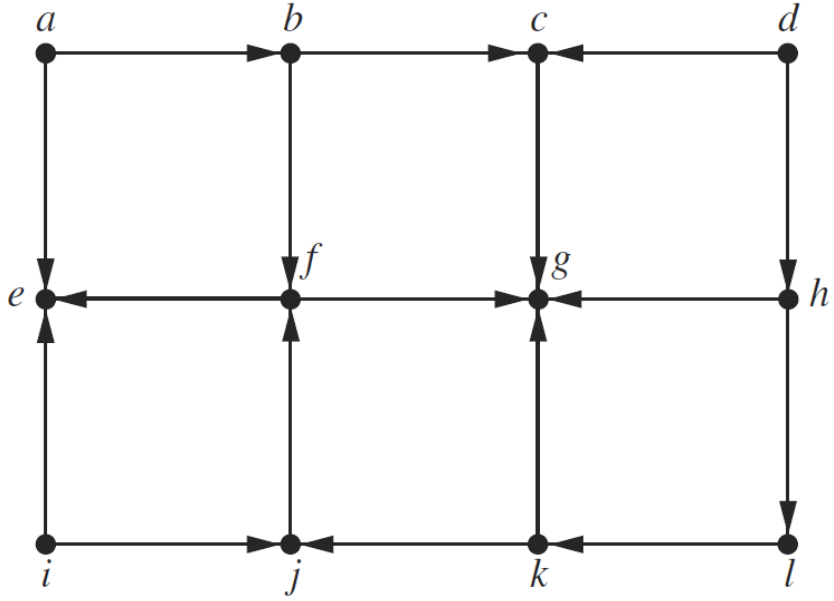


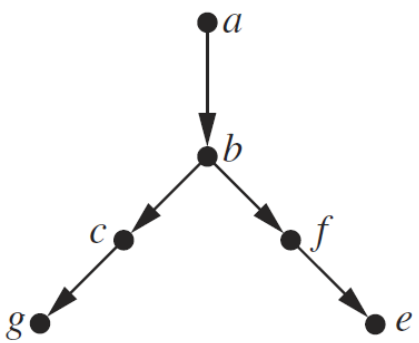
Using three colors to represent a graph

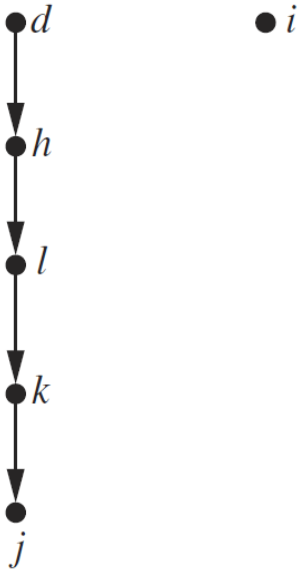


Depth-first search in directed graphs

- We can easily modify both depth-first search and breadth-first search so that they can run given a directed graph as input. However, the output will not necessarily be a spanning tree, but rather a spanning forest.



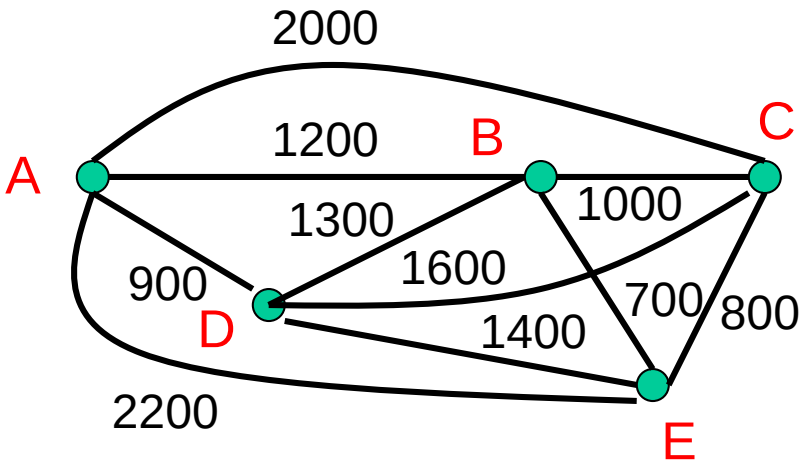




Exercises

- **Page 832: 11, 12, 15, 28, 33**

11.5 Minimum Spanning Trees



A weighted graph showing monthly lease costs for lines in a computer network.

Problem: Which links should be made to ensure that there is a path between ant two computer centers so that the total cost of the network is minimized?

We can solve this problem by finding a spanning tree so that the sum of the weights of the edges of the tree is minimized. Such a spanning tree is called a minimum spanning tree.

Algorithms for minimum spanning trees

- Definition 1: A minimum spanning tree in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.
- Two algorithms for constructing minimum spanning trees.

  Prim's algorithm

  Kruskal's algorithm
- Both proceed by successively adding edges of smallest weight from those edges with a specified property that have not already been used. These two algorithms are examples of greedy algorithms.

Prim's algorithm

Procedure *Prim* (*G*: weighted connected undirected graph with *n* vertices)

T:= a minimum-weight edge

for *i*:= 1 to *n*-2

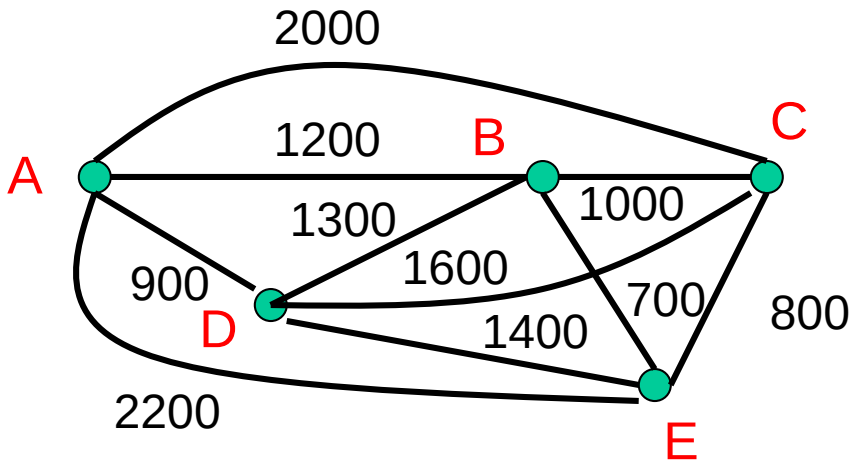
begin

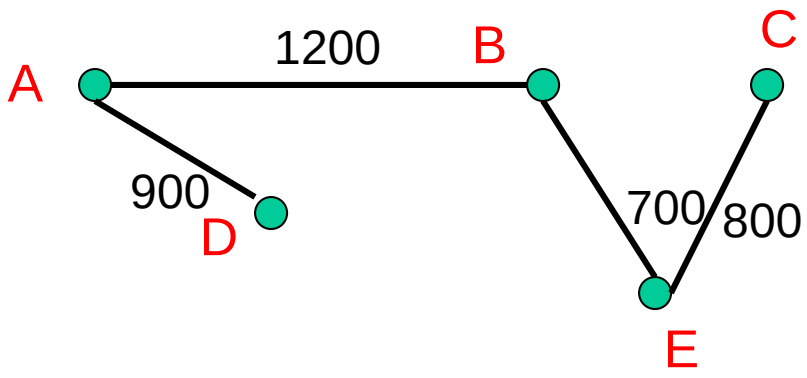
e:= an edge of minimum weight incident to a vertex in *T* and not forming a simple circuit in *T* if added to *T*.

T:= *T* with e added

end {*T* is a minimum spanning tree of *G*}

Example





Choice	Edge	Cost
1	BE	700
2	EC	800
3	BA	1200
4	AD	900
	Total:	3600

Kruskal's algorithm

procedure Kruskal (*G*: weighted connected undirected graph with n vertices)

T:= empty graph

for i:= 1 to *n*-1

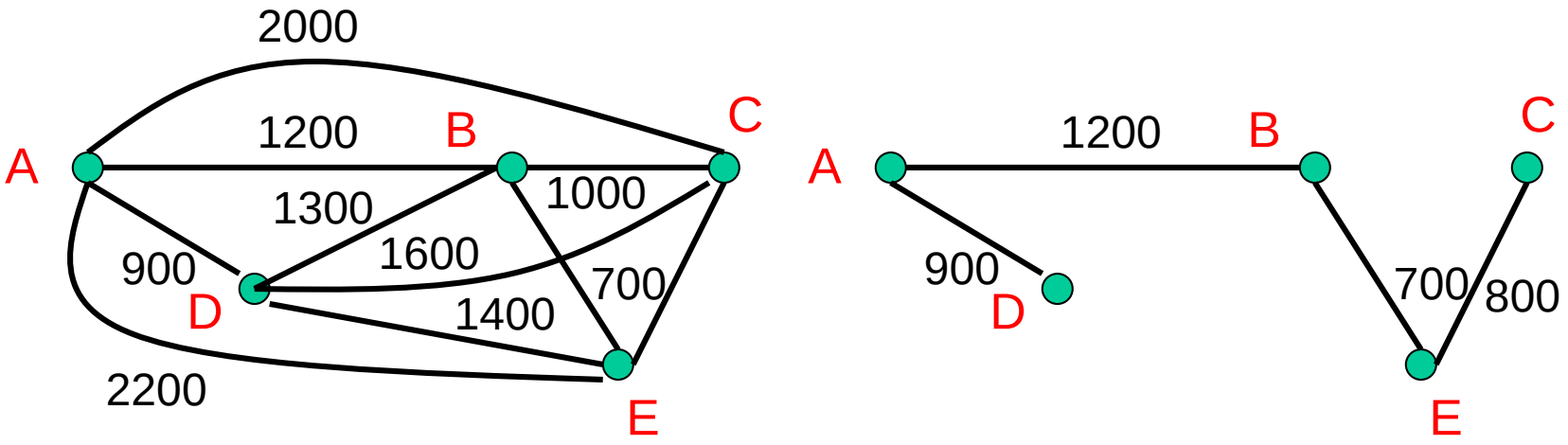
begin

e:= any edge in *G* with smallest weight that does not form a simple circuit when added to *T*

T:= *T* with e added

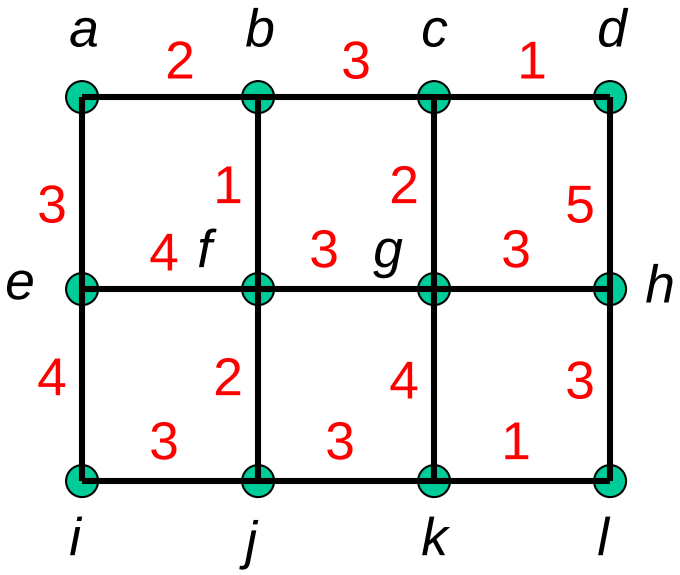
end {*T* is a minimum spanning tree of *G*}

Example



Choice	Edge	Cost
1	BE	700
2	EC	800
3	AD	900
4	AB	1200
	Total:	3600

Example



Choice	edge	weight	edge	weight
1	bf	1	cd	1
2	ab	2	k l	1
3	f j	2	bf	1
4	ae	3	cg	2
5	i j	3	ab	2
6	fg	3	f j	2
7	cg	2	bc	3
8	cd	1	jk	3
9	gh	3	gh	3
10	hl	3	l j	3
11	kl	1	ae	3

Notes

- Prim's algorithm: O (e log v)

  Kruskal's algorithm: O (e log e)
- The proofs that these two algorithms can produce a minimum spanning tree, are discussed in Page 838, and exercise 32 (page 803)

Exercises

- **Page 839: 3, 7, 14, 32**